JS

JavaScript desde cero hasta la Biblioteca

Guía DWEC · UD2

Teoría, 13 niveles de práctica con ejercicios resueltos, errores típicos y checklist de examen.

Índice

Cómo usar esta guía

PARTE I · TEORÍA

T1 · Herramientas, etiqueta script y modo estricto
T2 · Variables, tipos y cadenas
T3 · Operadores, valores falsy, ?? y ternario
T4 · if, switch y bucles
T5 · Arrays y objetos JSON
T6 · Funciones, closures e IIFE
T7 · Elementos predefinidos: conversión, Math, String, Date, JSON y temporizadores
T8 · Programación orientada a objetos
T9 · Módulos (export / import)
T10 · DOM y eventos (lo que usamos en los ejercicios)

PARTE II · PRÁCTICA

Nivel 1 · Variables, alert y pedir datos en bucle
Nivel 2 · Figuras con asteriscos
Nivel 3 · Menús, funciones como parámetro, tablas y factorial
Nivel 4 · arguments, contar en objetos y primos
Nivel 5 · Juegos con números aleatorios
Nivel 6 · Algoritmos del Refuerzo (sin métodos)
Nivel 7 · Arrays de objetos JSON y CRUD con menú
Nivel 8 · Closures y la IIFE $yedra (UD2E4)
Nivel 9 · Primeras páginas con DOM: Dados y Mercadillo
Nivel 10 · Temporizadores y el Simon (UD2E3)
Nivel 11 · POO en la práctica
Nivel 12 · La Juguetería completa, paso a paso
Nivel 13 · La Biblioteca: el ejercicio completo

PARTE III · ANTES DEL EXAMEN

Errores típicos (los que salieron en tus ejercicios)
Checklist para el examen

Cómo usar esta guía

Esta guía te lleva desde la primera línea de JavaScript hasta construir una aplicación completa como la Juguetería o la Biblioteca, usando solo lo que aparece en los apuntes de la UD2 y en los ejercicios que hemos resuelto (Lógica, Refuerzo 1, Mercadillo, Simon, Dados, $yedra, Juguetería y Biblioteca).

Orden de estudio. Lee la Parte I (teoría) una vez, sin memorizar. Después haz la Parte II nivel a nivel: cada nivel usa lo del anterior. No pases de nivel hasta que puedas hacer el ejercicio propuesto sin mirar la solución.

Método para cada ejercicio (el mismo que pide el profesor en el Simon):

1. **Entiende qué hay que construir.** Escribe en un comentario qué entra (parámetros, prompt) y qué sale (return, alert, HTML).
2. **Dibújalo en papel.** En las figuras, haz una tabla fila a fila. En las aplicaciones, dibuja las pantallas y qué botón lleva a cada una.
3. **Piensa los datos.** ¿Un número? ¿Un array? ¿Un array de objetos JSON? Elegir bien la estructura hace fácil el resto.
4. **Trocea.** Una función por tarea. Escribe primero los comentarios de las funciones y luego el código.
5. **Prueba cada trozo antes de seguir** (abajo cómo).

Cómo probar sin funciones de test. Abre la página con Live Server y pulsa F12:

- **Consola:** escribe el nombre de una función y llámala: `dibujarTriangulo(4)`. Ves el resultado al momento.
- **console.log:** pon `console.log('fila', i, linea)` dentro del bucle para ver cómo cambia cada variable.
- **Errores en rojo:** te dicen el fichero y la línea. Léelos siempre antes de tocar nada.
- **Pestaña Elementos:** comprueba que el HTML que has generado con `innerHTML` es el que esperabas (ids, `data-id`, `data-accion`).
- Las figuras compruébalas con `console.log`, no con `alert`: el alert usa una letra donde el espacio es más estrecho y la figura se ve torcida aunque esté bien.

Estilo que usamos en todo el código (el de tus ejercicios): `'use strict'`, `let` / `const`, comillas simples en JS y dobles en HTML, `===`, `for...in` para buscar dentro de objetos y arrays, `do...while` + `switch` para los menús, `prompt` / `alert` para hablar con el usuario, y en el HTML solo `div`, `label` para el texto de los campos e `input type="button"` para los botones.

PARTE I · TEORÍA

T1 · Herramientas, etiqueta script y modo estricto

Trabajas con VS Code + Live Server (sirve la página en `localhost:5500` y la recarga al guardar con Ctrl+S) y depuras con F12 en Chrome.

Estructura mínima de un ejercicio (HTML y JS separados, como recomiendan los apuntes):

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ejercicio</title>
    <script src="app.js" defer></script>
</head>
<body>
    <div id="resultado"></div>
</body>
</html>
```

```
'use strict';
console.log('Hola mundo!');
```

Lo que tienes que saber de memoria:

- `<script>` **siempre** lleva cierre `</script>`. `<script src="app.js" />` da error.
- `defer`: el script se descarga a la vez que el HTML pero se ejecuta cuando el HTML ya está leído entero. Sin `defer`, un script en el `<head>` no encuentra los elementos del `<body>`.
- `'use strict'` tiene que ser la **primera** instrucción del fichero (o de la función). Si no, no se activa.
- Con modo estricto: no puedes usar variables sin declarar, ni `delete` sobre variables, ni repetir nombres de parámetros, y `this` en una función suelta vale `undefined`.
- Comentarios: `//` una línea, `/* */` bloque, `/** */` el de VS Code con asteriscos.
- En la consola, `<` es lo que escribes y `>` lo que devuelve. Ctrl+L la limpia.

Tipos de mensaje en consola: `console.log`, `console.error`, `console.warn`, `console.info`, `console.debug`, `console.trace`. Todos aceptan varios valores: `console.log('a vale', a, 'b vale', b)`.

T2 · Variables, tipos y cadenas

Declara siempre con `let` (cambia) o `const` (no se puede reasignar). `var` no se usa. Una variable sin valor vale `undefined`.

```
let edad;            // undefined
edad = 23;
const IVA = 21;      // IVA = 10 daría error
```

```
let nombre = 'Ana';
nombre = 5;          // tipado débil: ahora es un número
```

Tipos (compruébalos con `typeof`):

Valor	typeof	Nota
`'hola'`	string	usa comillas simples en JS
`42`, `3.14`	number	enteros y decimales son el mismo tipo
`true`	boolean	
`undefined`	undefined	variable sin valor
`null`	object	**trampa de examen**: es un valor, pero typeof dice object
`[1, 2]`	object	un array es un objeto
`{ a: 1 }`	object	objeto JSON
`function(){}`	function	

Valores especiales: `NaN` (operación numérica que falla, `'abc' * 2`) e `Infinity` (`1 / 0`).

Trampas de tipos (salen en preguntas de "¿qué muestra?"):

```
console.log('70' - 5);   // 65   (el - convierte a número)
console.log('70' + 5);   // '705' (el + con una cadena concatena)
console.log(0.1 + 0.2);  // 0.30000000000000004
```

Por eso, todo lo que viene de `prompt` o de un `input.value` es **texto** y hay que convertirlo antes de sumar (`parseInt`, `parseFloat`, `Number`).

Cadenas. Tres formas: `'simple'` (la que usamos), `"doble"` (la dejamos para el HTML) y `` `plantilla` `` con `${}`:

```
let producto = 'balón';
let precio = 12;
let texto = `El ${producto} cuesta ${precio * 2} € por pareja`;
let mensaje = `Eres ${edad < 18 ? 'menor' : 'mayor'} de edad`;
```

Las plantillas respetan los saltos de línea, por eso son las que usamos para generar HTML. Escapes: `\n` nueva línea, `\t` tabulador, `\'` comilla simple, `\"` comilla doble, `\\` barra.

Una cadena es como un array de caracteres: `texto[0]` es la primera letra y `texto.length` su longitud. Esto es lo que usamos en el Refuerzo para recorrer palabras letra a letra sin métodos.

T3 · Operadores, valores falsy, ?? y ternario

Aritméticos: `+ - * / % **`. El `%` (resto) es el que más usarás: `n % 2 === 0` es par, `n % 3 === 0` es múltiplo de 3.

Incremento: `i++` devuelve el valor y luego suma; `++i` suma y luego devuelve.

```
let c = 10;
console.log(c++); // 10 (y ahora c vale 11)
```

```
console.log(++c); // 12
```

Asignación abreviada: `+= -= *= /= **=`. Cuidado: en las figuras se usa `linea += '*'`. Si escribes `linea *= '*'` obtienes `NaN` (error real de tus primeros triángulos).

Comparación: usa siempre `===` y `!==` (comparan valor **y** tipo).

```
10 == '10'   // true  (solo valor)
10 === '10'  // false (distinto tipo)
```

Lógicos: `&&` (y), `||` (o), `!` (no).

Valores falsy (cuentan como `false` en un `if`): `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. Todo lo demás es truthy, incluidos `'0'`, `[]` y `{}`.

Por eso funciona `if (!producto)` para saber si una búsqueda no encontró nada (`undefined`).

Cortocircuito:

- `a || b` devuelve el **primer truthy** (o el último si ninguno lo es).
- `a && b` devuelve el **primer falsy** (o el último si todos son truthy).
- `a ?? b` devuelve `b` **solo** si `a` es `null` o `undefined`.

Expresión	Resultado	Por qué
`0 || 21`	21	0 es falsy
`0 ?? 21`	0	0 no es null ni undefined
`'' || 'Anónimo'`	'Anónimo'	'' es falsy
`'' ?? 'Anónimo'`	''	'' no es null ni undefined
`undefined ?? 21`	21	
`'Juan' && 'Ana'`	'Ana'	ambos truthy, devuelve el último

Ternario: un if/else en una línea, para casos sencillos.

```
let tipo = numero >= 0 ? 'Positivo' : 'Negativo';
```

Encadenamiento opcional `?.` (aparece en POO): `persona.direccion?.ciudad` devuelve `undefined` en vez de dar error si `direccion` no existe.

T4 · if, switch y bucles

if / else if / else:

```
if (temperatura > 20) {
    console.log('Agradable');
} else if (temperatura > 10) {
    console.log('Fresco');
} else {
    console.log('Frío');
}
```

switch: compara con `===`. Sin `break` sigue ejecutando el siguiente `case`. `default` es el "si no coincide ninguno".

```
switch (opcion) {
    case '1':
        console.log('Cuadrado');
        break;
    case '2':
        console.log('Triángulo');
        break;
    default:
        alert('Opción no válida');
}
```

Ojo: `prompt` devuelve texto, así que los `case` van con comillas (`'1'`), o conviertes antes con `parseInt`.

Qué bucle elegir:

Bucle	Cuándo	Vueltas
`for (let i = 0; i < n; i++)`	sabes cuántas vueltas (figuras, tablas)	n
`for (let valor of array)`	quieres cada **valor** de un array	una por elemento
`for (let clave in objeto)`	quieres cada **clave** de un objeto o cada **índice** de un array	una por clave
`while (condición)`	repetir mientras se cumpla; puede no entrar	0 o más
`do { } while (condición)`	repetir al menos una vez: **menús y pedir datos**	1 o más

```
let alumno = { nombre: 'Willy', edad: 23 };
for (let clave in alumno) {
    console.log(clave, alumno[clave]); // nombre Willy / edad 23
}

let notas = [7, 4, 9];
for (let i in notas) {
    console.log(i, notas[i]); // '0' 7 / '1' 4 / '2' 9
}
for (let nota of notas) {
    console.log(nota); // 7 / 4 / 9
}
```

Trampa: con `for...in` sobre un array, `i` es un **texto** (`'0'`, `'1'`). Para operar con él (`i + 1`) conviértelo: `Number(i) + 1`.

break sale del bucle. **continue** salta a la siguiente vuelta.

```
for (let i = 0; i < 10; i++) {
    if (i === 5) break;    // termina en 5
    if (i === 3) continue; // se salta el 3
    console.log(i);        // 0 1 2 4
}
```

T5 · Arrays y objetos JSON

Casi todos los ejercicios grandes guardan los datos en **un array de objetos JSON** o en **un objeto cuyas claves son nombres**. Domina estas dos estructuras y el resto es recorrerlas.

Arrays

```
let numeros = [10, 20, 30];
numeros[0];                    // 10
numeros[numeros.length - 1];   // 30, el último
numeros[5] = 60;               // crea huecos undefined en 3 y 4
let vacio = new Array(4);      // 4 huecos undefined
let matriz = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];
matriz[1][1];                  // 5 (fila 1, columna 1)
```

Método	Qué hace	¿Cambia el array?
`push(x)`	añade al final, devuelve la nueva longitud	sí
`pop()`	quita el último y lo devuelve	sí
`shift()` / `unshift(x)`	quita / añade al principio	sí
`indexOf(x)`	posición de x, o -1 si no está	no
`splice(i, n)`	borra n elementos desde la posición i	sí
`splice(i, 0, a, b)`	inserta a y b en la posición i	sí
`slice(i, f)`	copia de i hasta f (sin incluir f)	no
`sort(fn)` / `toSorted(fn)`	ordena / devuelve copia ordenada	sí / no
`reverse()` / `toReversed()`	invierte / copia invertida	sí / no
`filter(fn)`	nuevo array con los que cumplen	no
`map(fn)`	nuevo array transformando cada uno	no
`forEach(fn)`	recorre (sin break ni continue)	no
`join(sep)`	une en una cadena	no
`concat(a2)`, `flat()`, `fill(v)`	une, aplana, rellena	no, no, sí

`slice` ≠ `splice`: `slice` copia y no toca el original; `splice` borra o inserta en el original. Confundirlos fue uno de tus fallos en `borrarLibro`.

Ordenar con criterio. La función recibe dos elementos `a` y `b` y devuelve negativo si `a` va antes, positivo si `b` va antes y 0 si da igual:

```
numeros.sort((a, b) => a - b);                 // ascendente
numeros.sort((a, b) => b - a);                 // descendente
productos.toSorted((a, b) => a.precio - b.precio);
alumnos.sort((a, b) => a.nombre.localeCompare(b.nombre)); // alfabético
```

Sin función, `sort()` ordena como texto: `[4, 55, 1, 11]` queda `[1, 11, 4, 55]`.

Copia: `let copia = [...original];` Si no copias, `let copia = original` es **el mismo** array con dos nombres.

Desestructuración: `let [a, b, ...resto] = [10, 20, 30, 40];` y `let { nombre, edad } = alumno;` (las variables van entre `[]` o `{}`).

Objetos JSON (clave-valor)

```
let persona = { nombre: 'Juan', edad: 30 };
persona.nombre;          // 'Juan'  notación punto
persona['edad'];         // 30      notación corchetes
persona.ciudad = 'Laredo'; // añade una propiedad nueva

let clave = 'nombre';
persona[clave];          // 'Juan'  la clave está en una variable
```

La **notación de corchetes con una variable** es la clave de muchos ejercicios: contar caracteres (`contador[letra]++`), agrupar por módulo (`grupos[alumno.modulo]`), el inventario del Mercadillo (`inventario[nombre]`).

- ¿Existe la clave? `if (inventario[nombre] === undefined)` → no existe.
- Recorrer: `for (let clave in objeto) { objeto[clave] }`.
- Los apuntes muestran `delete objeto.clave`, pero también dicen que en modo estricto no se usa `delete`. En tus ejercicios lo resolvemos creando un objeto nuevo sin esa clave (lo verás en el Mercadillo).

Array de objetos (la estructura de casi todo):

```
let alumnos = [
    { nombre: 'Ana', nota: 7, modulo: 'DWEC' },
    { nombre: 'Luis', nota: 3, modulo: 'DWES' }
];
alumnos[1].nombre;       // 'Luis'
for (let i in alumnos) {
    console.log(alumnos[i].nombre, alumnos[i].nota);
}
```

T6 · Funciones, closures e IIFE

Con nombre, con parámetros por defecto:

```
function saludar(nombre = 'desconocido') {
    return 'Hola ' + nombre;
}
saludar();        // 'Hola desconocido'
saludar('Ana');   // 'Hola Ana'
```

Un parámetro es solo un nombre para lo que te pasen al llamar. En `crearJuguete(nuevo)`, `nuevo` no existe antes: es el objeto que escribe quien llama, por ejemplo `crearJuguete({ nombre: 'Pelota', marca: 'Nike', precio: 10 })`.

Número variable de parámetros: `arguments` (solo en funciones normales, no es un array de verdad) o `...rest` (sí es un array).

```
function sumar(...numeros) {
    let total = 0;
    for (let n of numeros) {
        total += n;
    }
    return total;
}
sumar(1, 2, 3, 4); // 10
```

Anónima (guardada en una variable o pasada como parámetro) y **arrow**:

```
let saludo = function () { return 'Hola'; };
let doble = (n) => n * 2;          // return implícito
let suma = (a, b) => { return a + b; };
```

Funciones como parámetro (callback). Pasas la función **sin paréntesis**; quien la recibe la ejecuta con paréntesis. Es la base del ejercicio 6 (dibujar la figura que te pasen), de `sort`, `addEventListener` y `setTimeout`.

```
function mostrar(funcionFigura, tamano) {
    console.log(funcionFigura(tamano)); // aquí se ejecuta
}
mostrar(dibujarTriangulo, 4);          // se pasa sin ()
```

Si escribes `dibujarTriangulo()` al pasarla, la ejecutas antes y pasas su resultado. Y si te olvidas los `()` al **llamar** (`this.generarHTMLBuscador` en vez de `this.generarHTMLBuscador()`), en la página aparece el código de la función como texto: otro de tus errores en la Juguetería.

Arrow y `this`: una arrow no tiene `this` propio, lo hereda de donde se escribe. Regla de los apuntes: funciones normales para los métodos, arrow para los callbacks (eventos, temporizadores, `sort`).

Closure: una función que devuelve otra función. La de dentro recuerda las variables de la de fuera aunque esta ya haya terminado.

```
function crearContador() {
    let contador = 0;            // estado guardado
    return function () {
        contador++;
        return contador;
    };
}
let c1 = crearContador();
c1(); // 1
c1(); // 2
let c2 = crearContador();
c2(); // 1  (cada closure tiene su propio contador)
```

IIFE (función autoinvocada): se define y se ejecuta a la vez. Lo de dentro queda **privado** y solo se ve lo que devuelves en el objeto del `return`. Es el patrón de `$negocio`, `simon`, `$yedra` y `$biblio`.

```
const contador = (function () {
    let valor = 0;                        // privado
```

```
    function incrementar() { valor++; }
    function obtenerValor() { return valor; }
    return { incrementar, obtenerValor }; // lo público
})();

contador.incrementar();
contador.obtenerValor(); // 1
contador.valor;          // undefined: no se ve desde fuera
```

`return { incrementar, obtenerValor }` es lo mismo que `return { incrementar: incrementar, obtenerValor: obtenerValor }`.

Recursividad: una función que se llama a sí misma con un caso que la para.

```
function factorial(n) {
    if (n <= 1) {
        return 1;               // caso base: para
    }
    return n * factorial(n - 1); // 4 * 3 * 2 * 1
}
```

T7 · Elementos predefinidos: conversión, Math, String, Date, JSON y temporizadores

Convertir y validar números

```
parseInt('42');        // 42
parseInt('12.9');      // 12
parseFloat('3.14');    // 3.14
parseFloat('10abc');   // 10
parseInt('abc');       // NaN
Number('12px');        // NaN (más estricto)
isNaN(parseInt('hola')); // true  → dato no válido
```

Patrón para pedir un número válido (lo usarás en todos los ejercicios con `prompt`):

```
let n;
do {
    n = parseInt(prompt('Número entre 1 y 10'));
} while (isNaN(n) || n < 1 || n > 10);
```

Math

Método	Ejemplo	Resultado
`Math.floor(x)`	`Math.floor(4.8)`	4 (hacia abajo)
`Math.ceil(x)`	`Math.ceil(4.2)`	5 (hacia arriba)
`Math.round(x)`	`Math.round(4.5)`	5 (al más cercano)
`Math.trunc(x)`	`Math.trunc(4.9)`	4 (quita decimales)
`Math.max(...)`, `Math.min(...)`	`Math.max(1, 5, 10)`	10
`Math.abs`, `Math.sqrt`, `Math.pow`	`Math.sqrt(16)`	4
`Math.random()`		decimal en [0, 1)

Fórmula del aleatorio entero entre `min` y `max` (ambos incluidos) — Dados, Simon, adivinar número, mates:

```
let numero = Math.floor(Math.random() * (max - min + 1)) + min;
// entre 1 y 6:  Math.floor(Math.random() * 6) + 1
// índice de un array: Math.floor(Math.random() * colores.length)
```

Por qué funciona: `Math.random() * 6` da de 0 a 5,999...; `floor` lo deja en 0...5; `+ 1` lo pasa a 1...6.

Truncar a 2 decimales (como en $yedra): `Math.trunc(media * 100) / 100` → 6.6666 pasa a 666.66, se trunca a 666 y se divide: 6.66.

String (no modifican la cadena, devuelven otra)

`charAt(i)`, `indexOf(sub)`, `includes(sub)`, `slice(ini, fin)`, `replace(a, b)`, `split(sep)`, `toLowerCase()`, `toUpperCase()`, `trim()`, `localeCompare(otra)`.

Buscador sin importar mayúsculas (Juguetería): `juguete.nombre.toLowerCase().includes(filtro.toLowerCase())`.

Date

Los meses van de **0 a 11**. `new Date()` es ahora; `new Date(2023, 8, 30)` es 30 de septiembre.

Lectura: `getFullYear()`, `getMonth()`, `getDate()` (día del mes), `getDay()` (día de la semana, 0 = domingo), `getHours()`, `getMinutes()`, `getTime()`.

Restar fechas da milisegundos: `(fecha2 - fecha1) / (1000 * 60 * 60 * 24)` son días. Antigüedad de un trabajador: `new Date().getFullYear() - trabajador.contratacion`.

Fecha de hoy en texto (la usamos en los préstamos de la Biblioteca):

```
function fechaHoy() {
    let hoy = new Date();
    let mes = hoy.getMonth() + 1;   // +1 porque empieza en 0
    let dia = hoy.getDate();
    if (mes < 10) mes = '0' + mes;
    if (dia < 10) dia = '0' + dia;
```

```
    return hoy.getFullYear() + '-' + mes + '-' + dia; // '2026-11-03'
}
```

JSON

```
let texto = JSON.stringify(alumnos);   // array/objeto → cadena
let datos = JSON.parse(texto);         // cadena → array/objeto

try {
    let nuevos = JSON.parse(cadena);
    if (!Array.isArray(nuevos)) {
        alert('El JSON no es un array');
    }
} catch (error) {
    alert('JSON no válido: ' + error.message);
}
```

En la cadena JSON las claves y los textos van con comillas dobles: `'{"nombre":"Juan"}'`. Si faltan, `JSON.parse` lanza error (por eso el `try...catch`).

Temporizadores (los pone el navegador)

```
let id = setTimeout(() => console.log('a los 2 s'), 2000); // una vez
clearTimeout(id);

let intervalo = setInterval(() => console.log('cada 1 s'), 1000); // se repite
clearInterval(intervalo);
```

`setTimeout` **no para** el programa: el código de debajo se ejecuta enseguida y la función se ejecuta después. Por eso en el Simon se encadenan los `setTimeout` (cada uno lanza el siguiente).

Map y Set existen (colecciones clave-valor y de valores únicos), pero en los ejercicios usamos JSON y arrays: el Refuerzo prohíbe `Set` en el ejercicio de conjuntos.

T8 · Programación orientada a objetos

JavaScript tiene tres formas de crear objetos; las tres funcionan con **prototipos** y son equivalentes.

Forma	Para qué	Problema
Objeto JSON `{ }`	un objeto suelto, datos	repites la estructura en cada objeto
Función constructora + `new`	muchos objetos iguales (forma antigua)	los métodos se copian en cada objeto si no usas `prototype`
`class` (ES6)	la forma recomendada	—

Función constructora + prototype:

```
function Persona(nombre, edad) {
    this.nombre = nombre;   // propiedades en el constructor
    this.edad = edad;
```

```
}
Persona.prototype.saludar = function () {   // métodos en el prototype
    console.log('Hola, soy ' + this.nombre);
};
let p = new Persona('Ana', 25);
p.saludar();
```

Con `prototype` todas las personas comparten **un solo** `saludar` (ahorra memoria). Si lo pones como `this.saludar = function…` dentro del constructor, cada objeto tiene su copia.

Clase ES6 completa (todo lo que entra en el examen en un ejemplo):

```
class Producto {
    static IVA = 21;            // estático: de la clase, no del objeto
    #nombre;                    // privados: declarados ANTES del constructor
    #precio;

    constructor(nombre = '', precio = 0) {
        this.#nombre = nombre;
        this.#precio = precio;
    }

    get nombre() {              // getter: se usa como variable
        return this.#nombre;
    }
    set nombre(valor) {         // setter: puede validar
        if (valor) {
            this.#nombre = valor;
        }
    }
    get precioConIva() {        // propiedad calculada
        return this.#precio * (1 + Producto.IVA / 100);
    }

    describir() {               // método normal
        return `${this.#nombre}: ${this.precioConIva} €`;
    }

    static crearGratis() {      // método estático
        return new Producto('Regalo', 0);
    }
}

let balon = new Producto('Balón', 10);
balon.nombre;            // 'Balón'   (sin paréntesis: es un getter)
balon.nombre = 'Pelota'; // llama al setter
balon.precioConIva;      // 12.1
balon.#precio;           // ERROR: es privado
Producto.IVA;            // 21 (se accede por la clase)
Producto.crearGratis();
```

Trampa que te pasó en `libro.js`: si declaras un campo público con el **mismo nombre** que un getter (`titulo;` arriba y `get titulo()` abajo), el campo tapa al getter y lees `undefined`. Los campos que declares arriba deben ser los privados con `#`.

Herencia: `extends` para heredar y `super` para llamar al padre. Solo un padre.

```
class Estudiante extends Persona {
    constructor(nombre, curso) {
        super(nombre);          // primero el constructor del padre
        this.curso = curso;
    }
    estudiar() {
        console.log(`${this.nombre} estudia ${this.curso}`);
    }
}
```

`this`: en un método normal es el objeto que lo llama. Si pasas un método como callback (`setTimeout(objeto.metodo, 1000)`) pierde su `this`. Dentro de una clase, los callbacks de eventos se escriben como arrow para conservar el `this` de la clase:

```
this.#contenedor.addEventListener('click', (evento) => {
    this.borrarJuguete(id);   // this sigue siendo la Juguetería
});
```

`Object.keys(obj)`, `Object.values(obj)` y `Object.entries(obj)` dan claves, valores y pares de un objeto (están en los apuntes, aunque en los ejercicios usamos `for...in`).

T9 · Módulos (export / import)

Un módulo es un fichero `.js` que **exporta** lo que quiere compartir; otro fichero lo **importa**. Así se reparte la Juguetería en `datos.js`, `juguete.js` y `jugueteria.js`.

```
// juguete.js
class Juguete { /* ... */ }
export default Juguete;          // UNO por fichero, se importa SIN llaves

// utilidades.js
const IVA = 21;
function redondear(n) { return Math.round(n * 100) / 100; }
export { IVA, redondear };        // nombrados, se importan CON llaves

// jugueteria.js
import Juguete from './juguete.js';
import { IVA, redondear } from './utilidades.js';
import * as Utils from './utilidades.js'; // todo junto: Utils.IVA
import { redondear as r } from './utilidades.js'; // con alias
```

En el HTML, el script que importa lleva `type="module"` (sin él: *Cannot use import statement outside a module*):

```
<script type="module" src="js/main.js"></script>
```

Lo que tienes que recordar:

- Los módulos van siempre en **modo estricto** y se cargan diferidos (como `defer`), aunque no lo pongas.
- Cada módulo tiene su ámbito: una variable de un módulo **no existe** en otro si no la exportas e importas.

- La ruta empieza por `./` y lleva `.js`. Si la carpeta no coincide, en la consola sale un 404.
- Los módulos no funcionan abriendo el HTML con doble clic: necesitas Live Server.
- Como el módulo tiene su ámbito, desde la consola F12 no ves sus funciones. Para probar, haz `console.log` dentro del módulo o asigna temporalmente `window.$biblio = $biblio;`.

T10 · DOM y eventos (lo que usamos en los ejercicios)

Los apuntes de teoría casi no tratan el DOM; estas herramientas vienen de los enunciados del Mercadillo, Simon, Dados, Juguetería y Biblioteca. Con ellas se hacen todas las páginas.

Herramienta	Qué hace	Ejemplo
`document.getElementById('id')`	devuelve el elemento con ese id	`document.getElementById('nombre')`
`document.querySelector('selector')`	el primer elemento que cumple un selector CSS	`'#app'`, `'.boton.verde'`, `'[data-componente="listado"]'`
`elemento.value`	lo escrito en un `input` (siempre texto)	`parseInt(cantidad.value)`
`elemento.innerHTML`	lee o sustituye el HTML de dentro	`resultado.innerHTML = '<div>Hola</div>'`
`elemento.addEventListener('click', fn)`	ejecuta `fn` al hacer clic	
`window.addEventListener('load', fn)`	ejecuta `fn` cuando la página ha cargado	aquí se asignan los eventos
`elemento.className`	cambia las clases	Simon: `'boton verde encendido'`
`elemento.dataset.xxx`	lee el atributo `data-xxx`	`data-id="3"` → `dataset.id` vale `'3'`
`elemento.closest('selector')`	sube por los padres hasta el primero que cumple el selector	de un botón a su fila `[data-id]`

Esqueleto de cualquier página con eventos (Mercadillo, Dados, Cambio, César):

```
<div>
    <label>Número:</label>
    <input type="text" id="numero">
    <input type="button" id="btnCalcular" value="Calcular">
</div>
<div id="resultado"></div>
```

```
'use strict';
function calcular() {
    let n = parseInt(document.getElementById('numero').value);
    if (isNaN(n)) {
```

```
        alert('Escribe un número');
        return;
    }
    document.getElementById('resultado').innerHTML = `El doble es ${n * 2}`;
}

window.addEventListener('load', function () {
    document.getElementById('btnCalcular').addEventListener('click', calcular);
});
```

Por qué en `load`: si el script se ejecuta antes de que exista el botón, `getElementById` devuelve `null` y `null.addEventListener` da error.

Delegación de eventos con `dataset` y `closest` (Juguetería y Biblioteca). En vez de poner un evento a cada botón, se pone **uno** en el contenedor y se mira qué se ha pulsado:

```
<div data-componente="listado">
    <div data-id="3">
        <div>Pelota</div>
        <input type="button" value="Ver" data-accion="ver">
        <input type="button" value="Borrar" data-accion="borrar">
    </div>
</div>
```

```
listado.addEventListener('click', (evento) => {
    let boton = evento.target.closest('[data-accion]'); // ¿qué botón?
    if (!boton) return;                                 // clic en un hueco
    let fila = boton.closest('[data-id]');              // ¿de qué fila?
    let id = Number(fila.dataset.id);                   // '3' → 3
    switch (boton.dataset.accion) {
        case 'ver':    /* mostrar el 3 */ break;
        case 'borrar': /* borrar el 3 */  break;
    }
});
```

Camino de un clic en "Borrar": el clic **sube** del botón a la fila y de la fila al listado → el listado lo recoge → `evento.target` es el botón → `closest('[data-accion]')` da el botón → `closest('[data-id]')` da la fila → `dataset.id` da `'3'`.

Dos detalles que fallan mucho: `dataset` siempre da **texto** (convierte con `Number` antes de comparar con `===`), y el nombre de `data-accion="borrar"` tiene que escribirse igual en el `case 'borrar'`.

Cada vez que cambias `innerHTML` los elementos de dentro se crean de nuevo y pierden sus eventos. Por eso las aplicaciones tienen una función `mostrar(html)` que pinta y vuelve a llamar a `asignarEventos()`.

PARTE II · PRÁCTICA

Nivel 1 · Variables, alert y pedir datos en bucle

Objetivo: guardar datos en variables, mostrarlos con formato y pedir datos al usuario hasta que diga basta.

Ejemplo resuelto 1 · Variables y concatenación (Lógica, ej. 1)

Qué piden: 4 variables (2 cadenas y 2 números) y varios `alert`. Las cadenas entre comillas dobles y los números entre simples.

```
'use strict';
let nombre = 'Roberto';
let apellido = 'Sainz';
let edad = 20;
let anioNacimiento = 2006;

alert(`nombre: "${nombre}"\napellido: "${apellido}"\nedad: '${edad}'\nanioNacimiento:
'${anioNacimiento}'`);
alert(nombre + '\n' + apellido);
alert(edad + anioNacimiento);                    // 2026
alert(nombre + apellido + edad + anioNacimiento);  // 'RobertoSainz202006'
```

Entiende el último alert. El `+` se evalúa de izquierda a derecha:

Paso	Operación	Resultado	Tipo
1	`'Roberto' + 'Sainz'`	`'RobertoSainz'`	texto
2	`'RobertoSainz' + 20`	`'RobertoSainz20'`	texto (texto + número = concatenar)
3	`'RobertoSainz20' + 2006`	`'RobertoSainz202006'`	texto

Si cambias el orden, `edad + anioNacimiento + nombre + apellido` da `'2026RobertoSainz'`: primero suma los números y luego concatena. Esta pregunta es típica de examen.

La plantilla `` ` ` `` deja meter comillas dobles y simples sin escaparlas, y `\n` hace el salto de línea dentro del alert.

Ejemplo resuelto 2 · Pedir números hasta un 0 (Lógica, ej. 2)

Qué piden: pedir números hasta que escriba 0 y mostrar máximo, mínimo, suma, media y cuántos.

Antes de programar, piensa:

- Repito "pedir número" al menos una vez → `do...while (numero !== 0)`.
- Lo que viene de `prompt` es texto → `parseFloat` + `isNaN` para validar.
- Máximo y mínimo: el **primer** número válido es a la vez máximo y mínimo; después comparo cada nuevo con ellos. Uso `total === 0` para saber que es el primero.
- El 0 no cuenta como número de la lista.

```
function estadisticas() {
    let maximo;
    let minimo;
    let suma = 0;
    let total = 0;
    let numero;
    do {
        numero = parseFloat(prompt('Introduce un número (0 para terminar)'));
        if (isNaN(numero)) {
            alert('Eso no es un número');
        } else if (numero !== 0) {
            if (total === 0 || numero > maximo) {
                maximo = numero;
            }
            if (total === 0 || numero < minimo) {
                minimo = numero;
            }
            suma += numero;
            total++;
        }
    } while (numero !== 0);

    if (total === 0) {
        alert('No has introducido ningún número');
    } else {
        alert(`Máximo: ${maximo}\nMínimo: ${minimo}\nSuma: ${suma}\nMedia: ${suma /
total}\nTotal: ${total}`);
    }
}
estadisticas();
```

Traza con las entradas `5`, `hola`, `2`, `9`, `0`:

Entrada	numero	¿Qué pasa?	maximo	minimo	suma	total
5	5	primero: es máx y mín	5	5	5	1
hola	NaN	alert "no es un número"	5	5	5	1
2	2	2 < 5 → nuevo mínimo	5	2	7	2
9	9	9 > 5 → nuevo máximo	9	2	16	3
0	0	no entra, el while termina	9	2	16	3

Resultado: máximo 9, mínimo 2, suma 16, media 5.33, total 3.

Fíjate: con `NaN` el `while (numero !== 0)` sigue (NaN no es 0), así que vuelve a pedir. Bien.

Ejercicio propuesto 1

Pide notas (0 a 10) hasta que escriba `-1`. Muestra cuántas son aprobados, cuántas suspensos y la nota media. Rechaza con un alert las notas fuera de rango.

Solución:

```
function notas() {
    let aprobados = 0;
    let suspensos = 0;
    let suma = 0;
```

```
    let nota;
    do {
        nota = parseFloat(prompt('Nota de 0 a 10 (-1 para terminar)'));
        if (isNaN(nota) || nota < -1 || nota > 10) {
            alert('Nota no válida');
        } else if (nota !== -1) {
            if (nota >= 5) {
                aprobados++;
            } else {
                suspensos++;
            }
            suma += nota;
        }
    } while (nota !== -1);
    let total = aprobados + suspensos;
    alert(`Aprobados: ${aprobados}\nSuspensos: ${suspensos}\nMedia: ${total === 0 ? 0 : suma
/ total}`);
}
```

Nivel 2 · Figuras con asteriscos

Todas las figuras se resuelven igual: **una figura es una cuadrícula de filas y columnas**, y en cada casilla decides si va `*` o espacio. Un bucle recorre las filas; dentro, uno o varios bucles construyen la línea; al acabar la fila, guardas la línea.

El método (hazlo en papel antes de escribir código)

1. Dibuja la figura para un tamaño pequeño (4 o 5) en papel cuadriculado.
2. Numera las filas (y las columnas si hace falta).
3. Haz una tabla: para cada fila, **cuántos espacios** y **cuántos asteriscos**.
4. Busca la fórmula que relaciona el número de fila con esas cantidades.
5. Escribe un `for` de filas y, dentro, un `for` por cada tramo (espacios, asteriscos…).

Plantilla que usan todas (el enunciado pide devolver un array con las líneas):

```
function figura(n) {
    let resultado = [];
    for (let fila = 1; fila <= n; fila++) {
        let linea = '';
        // tramo 1: espacios     → for (...) linea += ' ';
        // tramo 2: asteriscos   → for (...) linea += '*';
        resultado.push(linea);
    }
    return resultado;
}

// Para verla (en la consola, no en el alert)
function mostrarFigura(figura) {
    let texto = '';
    for (let linea of figura) {
        texto += linea + '\n';
    }
    console.log(texto);
}
mostrarFigura(figura(4));
```

Figura 1 · Cuadrado lleno

Todas las filas iguales: n asteriscos. Dos bucles anidados, nada más.

```
function cuadradoLleno(n) {
    let figura = [];
    for (let fila = 0; fila < n; fila++) {
        let linea = '';
        for (let col = 0; col < n; col++) {
            linea += '*';
        }
        figura.push(linea);
    }
    return figura;
}
```

Figura 2 · Cuadrado hueco (Lógica, ej. 3)

Dibújalo con las coordenadas (fila, columna) para n = 5:

```
        col: 0 1 2 3 4
fila 0:      * * * * *   ← primera fila: todo *
fila 1:      *       *   ← solo col 0 y col 4
fila 2:      *       *
fila 3:      *       *
fila 4:      * * * * *   ← última fila: todo *
```

Regla: va `*` si estás en el **borde**: fila 0, fila n-1, columna 0 o columna n-1. Si no, espacio.

```
function cuadradoHueco(n) {
    let figura = [];
    for (let fila = 0; fila < n; fila++) {
        let linea = '';
        for (let col = 0; col < n; col++) {
            if (fila === 0 || fila === n - 1 || col === 0 || col === n - 1) {
                linea += '*';
            } else {
                linea += ' ';
            }
        }
        figura.push(linea);
    }
    return figura;
}
```

Figura 3 · Triángulo rectángulo

```
*       fila 1 → 1 asterisco
**      fila 2 → 2
***     fila 3 → 3
****    fila 4 → 4
```

Fórmula: en la fila `fila` hay `fila` asteriscos. Empezando en 1 se ve más claro.

```
function trianguloRectangulo(n) {
    let figura = [];
    for (let fila = 1; fila <= n; fila++) {
        let linea = '';
        for (let col = 1; col <= fila; col++) {
            linea += '*';
        }
        figura.push(linea);
    }
    return figura;
}
```

Figura 4 · Triángulo centrado o pirámide (Lógica, ej. 4)

Dibujo para n = 4 (los puntos son espacios):

```
...*
..***
.*****
*******
```

fila	espacios	asteriscos
1	3	1
2	2	3
3	1	5
4	0	7
fórmula	**n - fila**	**2·fila - 1**

Cómo sacas las fórmulas: los espacios bajan de 1 en 1 desde n-1 hasta 0 → `n - fila`. Los asteriscos suben de 2 en 2 empezando en 1 → son los impares → `2 * fila - 1`. Comprueba con la fila 3: 4-3 = 1 espacio, 2·3-1 = 5 asteriscos. Bien.

```
function triangulo(n) {
    let figura = [];
    for (let fila = 1; fila <= n; fila++) {
        let linea = '';
        for (let e = 1; e <= n - fila; e++) {
            linea += ' ';
        }
        for (let a = 1; a <= 2 * fila - 1; a++) {
            linea += '*';
        }
        figura.push(linea);
    }
    return figura;
}
```

Los dos errores que tuviste aquí: escribir `linea *= '*'` (da `NaN`) y poner `n - 1` en vez de `n - fila` en los espacios (todas las filas con los mismos espacios: sale torcido).

Figura 5 · Rombo (Lógica, ej. 5)

Un rombo de diagonal 7 es **una pirámide de 4 filas + la misma pirámide al revés sin repetir la fila central**:

```
...*      fila 1   ┐
..***     fila 2   │ pirámide de mitad = (7+1)/2 = 4 filas
.*****    fila 3   │
*******   fila 4   ┘ fila central (la diagonal: 7)
.*****    fila 3   ┐
..***     fila 2   │ las filas 3, 2, 1 otra vez (bucle hacia atrás)
...*      fila 1   ┘
```

Las fórmulas son las de la pirámide cambiando `n` por `mitad`: espacios `mitad - fila`, asteriscos `2·fila - 1`. Funciona con diagonal impar; valida que lo sea (`diagonal % 2 !== 0`).

```
function rombo(diagonal) {
    let figura = [];
    let mitad = (diagonal + 1) / 2;
    // arriba, incluida la fila central
    for (let fila = 1; fila <= mitad; fila++) {
        let linea = '';
        for (let e = 1; e <= mitad - fila; e++) {
            linea += ' ';
        }
        for (let a = 1; a <= 2 * fila - 1; a++) {
            linea += '*';
        }
        figura.push(linea);
    }
    // abajo: de mitad-1 hasta 1
    for (let fila = mitad - 1; fila >= 1; fila--) {
        let linea = '';
        for (let e = 1; e <= mitad - fila; e++) {
            linea += ' ';
        }
        for (let a = 1; a <= 2 * fila - 1; a++) {
            linea += '*';
        }
        figura.push(linea);
    }
    return figura;
}
```

Figura 6 · La escalera (Refuerzo, ej. 12)

Para 4 escalones (ascendente):

```
...._      fila 0: 4 espacios + '_'      (el peldaño de arriba)
..._|      fila 1: 3 espacios + '_|'
.._|       fila 2: 2 espacios + '_|'
._|        fila 3: 1 espacio  + '_|'
_|         fila 4: 0 espacios + '_|'
```

La fila 0 es especial (n espacios + `_`). Las demás: `n - fila` espacios + `_|`. Si n es negativo, la escalera baja: fila 0 es `_` y la fila k lleva `k - 1` espacios + `|_`. Si n es 0, `__`.

```
function escalera(n) {
    let figura = [];
    if (n === 0) {
        figura.push('__');
    } else if (n > 0) {
        let linea = '';
        for (let e = 1; e <= n; e++) linea += ' ';
        figura.push(linea + '_');
        for (let fila = 1; fila <= n; fila++) {
            linea = '';
            for (let e = 1; e <= n - fila; e++) linea += ' ';
            figura.push(linea + '_|');
        }
    } else {
        let escalones = -n;
        figura.push('_');
        for (let fila = 1; fila <= escalones; fila++) {
            let linea = '';
            for (let e = 1; e <= fila - 1; e++) linea += ' ';
            figura.push(linea + '|_');
        }
    }
    return figura;
}
```

Figura 7 · Logo de Zelda (Refuerzo, ej. 13)

Tres pirámides de n filas. Para n = 3 cada base mide 2·3-1 = 5 y las dos de abajo van separadas por 1 espacio: ancho total 11.

```
.....*          arriba: centro en la columna 2n-1 = 5
....***
...*****
..*.....*       abajo: pirámide izq. + hueco + pirámide der.
.***...***
*****.*****
```

Parte (fila i desde 0)	espacios	asteriscos	hueco central
Arriba	2n - 1 - i	2i + 1	—
Abajo	n - 1 - i	2i + 1 (dos veces)	2n - 1 - 2i

El hueco central se saca del dibujo: con n = 3 vale 5, 3, 1 (baja de 2 en 2 desde 2n-1).

```
function zelda(n) {
    let figura = [];
    for (let i = 0; i < n; i++) {
        let linea = '';
        for (let e = 0; e < 2 * n - 1 - i; e++) linea += ' ';
        for (let a = 0; a < 2 * i + 1; a++) linea += '*';
        figura.push(linea);
    }
    for (let i = 0; i < n; i++) {
        let linea = '';
        for (let e = 0; e < n - 1 - i; e++) linea += ' ';
        for (let a = 0; a < 2 * i + 1; a++) linea += '*';
```

```
        for (let e = 0; e < 2 * n - 1 - 2 * i; e++) linea += ' ';
        for (let a = 0; a < 2 * i + 1; a++) linea += '*';
        figura.push(linea);
    }
    return figura;
}
```

Figura 8 · Marco para frases (Refuerzo, ej. 9)

```
**********
* ¿Qué   *
* te     *
* parece *     ← la palabra más larga (6) marca el ancho: 6 + 4 = 10
* el     *
* reto?  *
**********
```

Pasos: (1) separar palabras, (2) buscar la más larga, (3) dibujar: borde de `mayor + 4` asteriscos; cada fila `'* ' + palabra + espacios de relleno + ' *'`. En el Refuerzo no se pueden usar métodos, así que separamos a mano recorriendo letra a letra y añadimos con `palabras[palabras.length] = palabra` (equivale a un `push`).

```
function marcoFrase(frase) {
    let palabras = [];
    let palabra = '';
    for (let i = 0; i < frase.length; i++) {
        if (frase[i] === ' ') {
            if (palabra !== '') {
                palabras[palabras.length] = palabra;
                palabra = '';
            }
        } else {
            palabra += frase[i];
        }
    }
    if (palabra !== '') {
        palabras[palabras.length] = palabra;   // la última no tiene espacio detrás
    }

    let mayor = 0;
    for (let i = 0; i < palabras.length; i++) {
        if (palabras[i].length > mayor) {
            mayor = palabras[i].length;
        }
    }

    let borde = '';
    for (let i = 0; i < mayor + 4; i++) borde += '*';
    let resultado = borde + '\n';
    for (let i = 0; i < palabras.length; i++) {
        let linea = '* ' + palabras[i];
        for (let e = palabras[i].length; e < mayor; e++) linea += ' ';
        resultado += linea + ' *\n';
    }
    resultado += borde;
    return resultado;
}
```

Ejercicio propuesto 2 · Reloj de arena

Para n = 4 (filas de 7, 5, 3, 1, 3, 5, 7 asteriscos). Pista: es el rombo **al revés**: primero la pirámide de n hacia atrás (de n a 1) y luego hacia delante sin repetir la fila de 1 (de 2 a n).

```
*******
.*****
..***
...*
..***
.*****
*******
```

Solución:

```
function relojArena(n) {
    let figura = [];
    for (let fila = n; fila >= 1; fila--) {
        let linea = '';
        for (let e = 1; e <= n - fila; e++) linea += ' ';
        for (let a = 1; a <= 2 * fila - 1; a++) linea += '*';
        figura.push(linea);
    }
    for (let fila = 2; fila <= n; fila++) {
        let linea = '';
        for (let e = 1; e <= n - fila; e++) linea += ' ';
        for (let a = 1; a <= 2 * fila - 1; a++) linea += '*';
        figura.push(linea);
    }
    return figura;
}
```

Todas las figuras de este nivel están probadas: la salida es exactamente la de los dibujos.

Nivel 3 · Menús, funciones como parámetro, tablas y factorial

Objetivo: el **menú con** `do...while` **+** `switch`, que es el esqueleto de los ejercicios 6, 13, 15 y de cualquier programa "con opción de terminar".

El esqueleto de un menú

```
function menu() {
    let opcion;
    do {
        opcion = prompt('1. Hacer A\n2. Hacer B\n0. Terminar');
        switch (opcion) {
            case '1':
                hacerA();
                break;
            case '2':
                hacerB();
                break;
            case '0':
                break;                      // no hace nada: el while terminará
            default:
```

```
                alert('Opción no válida');
        }
    } while (opcion !== '0');
}
```

Tres cosas: el `do` asegura que el menú sale al menos una vez; los `case` van entre comillas porque `prompt` devuelve texto; y si el usuario pulsa Cancelar, `prompt` devuelve `null` → cae en `default` y vuelve a preguntar.

Ejemplo resuelto 1 · Dibujar la figura elegida (Lógica, ej. 6)

Qué piden: una función que reciba **la función que dibuja** y el tamaño, y la muestre en un alert. Un menú elige la figura.

La idea clave: `dibujarFigura(triangulo, 4)` pasa la función **sin paréntesis**. Dentro, `funcionFigura(tamano)` la ejecuta. Así una sola función sirve para todas las figuras del nivel 2.

```
function dibujarFigura(funcionFigura, tamano) {
    let figura = funcionFigura(tamano);   // aquí se ejecuta triangulo(4), rombo(7)...
    let texto = '';
    for (let linea of figura) {
        texto += linea + '\n';
    }
    alert(texto);
}

function menuFiguras() {
    let opcion;
    do {
        opcion = prompt('1. Cuadrado hueco\n2. Triángulo\n3. Rombo\n0. Terminar');
        if (opcion === '1' || opcion === '2' || opcion === '3') {
            let tamano;
            do {
                tamano = parseInt(prompt('Tamaño (mayor que 0)'));
            } while (isNaN(tamano) || tamano <= 0);

            switch (opcion) {
                case '1':
                    dibujarFigura(cuadradoHueco, tamano);
                    break;
                case '2':
                    dibujarFigura(triangulo, tamano);
                    break;
                case '3':
                    dibujarFigura(rombo, tamano);
                    break;
            }
        } else if (opcion !== '0') {
            alert('Opción no válida');
        }
    } while (opcion !== '0');
}
menuFiguras();
```

El tamaño se pide **antes** del `switch` y una sola vez, porque las tres figuras lo necesitan. Así no repites el `do...while` tres veces.

Ejemplo resuelto 2 · Tablas de multiplicar (Lógica, ej. 7)

Qué piden: (a) función que imprime la tabla de un número; (b) función que pide dos números de 0 a 10 e imprime las tablas desde el menor hasta el mayor, aunque los escriba al revés.

Piensa: si me dan 7 y 3, tengo que ir de 3 a 7 → si `a > b`, los intercambio con una variable auxiliar.

```
function tablaMultiplicar(numero) {
    let texto = `Tabla del ${numero}\n`;
    for (let i = 0; i <= 10; i++) {
        texto += `${numero} x ${i} = ${numero * i}\n`;
    }
    return texto;
}

function pedirNumero(mensaje) {
    let n;
    do {
        n = parseInt(prompt(mensaje));
    } while (isNaN(n) || n < 0 || n > 10);
    return n;
}

function tablasEntre() {
    let a = pedirNumero('Primer número (0-10)');
    let b = pedirNumero('Segundo número (0-10)');
    if (a > b) {          // intercambio
        let aux = a;
        a = b;
        b = aux;
    }
    let texto = '';
    for (let n = a; n <= b; n++) {
        texto += tablaMultiplicar(n) + '\n';
    }
    console.log(texto);
}
```

El intercambio paso a paso con a = 7, b = 3: `aux = 7` → `a = 3` → `b = 7`. Sin `aux`, al hacer `a = b` perderías el 7.

Ejemplo resuelto 3 · Factorial con desarrollo (Lógica, ej. 8)

Qué piden: `factorial(4)` devuelve la cadena `'4x3x2x1=24'`. El script pide números hasta que el usuario no quiera seguir.

Piensa: un bucle de n hasta 1 que a la vez **multiplica** (`resultado *= i`) y **escribe** (`texto += i`). La `x` va entre números, no después del último → solo si `i > 1`.

```
function factorial(n) {
    let resultado = 1;
    let texto = '';
    for (let i = n; i >= 1; i--) {
        resultado *= i;
        texto += i;
        if (i > 1) {
            texto += 'x';
```

```
        }
    }
    return texto + '=' + resultado;
}

function probarFactorial() {
    let seguir;
    do {
        let n;
        do {
            n = parseInt(prompt('Número para el factorial (1 o más)'));
        } while (isNaN(n) || n < 1);
        alert(factorial(n));
        seguir = prompt('¿Otro? (s/n)');
    } while (seguir === 's');
}
```

i	resultado	texto
4	4	`'4x'`
3	12	`'4x3x'`
2	24	`'4x3x2x'`
1	24	`'4x3x2x1'` (sin x)
fin		`'4x3x2x1=24'`

En vez de `confirm`, preguntamos con `prompt` y comprobamos `'s'`: así lo hemos hecho en tus ejercicios.

Versión recursiva (Refuerzo, ej. 5): ver T6. `factorialRecursivo(5)` → `5 * f(4)` → `5 * 4 * f(3)` … → `5 * 4 * 3 * 2 * 1` = 120.

Ejercicio propuesto 3 · Calculadora con menú

Menú: 1 sumar, 2 restar, 3 multiplicar, 4 dividir, 0 salir. Pide dos números válidos y muestra el resultado. Si divide entre 0, avisa.

Solución:

```
function pedir(mensaje) {
    let n;
    do {
        n = parseFloat(prompt(mensaje));
    } while (isNaN(n));
    return n;
}

function calculadora() {
    let opcion;
    do {
        opcion = prompt('1. Sumar\n2. Restar\n3. Multiplicar\n4. Dividir\n0. Salir');
        if (opcion === '1' || opcion === '2' || opcion === '3' || opcion === '4') {
            let a = pedir('Primer número');
            let b = pedir('Segundo número');
            switch (opcion) {
                case '1':
                    alert(a + b);
```

```
                    break;
                case '2':
                    alert(a - b);
                    break;
                case '3':
                    alert(a * b);
                    break;
                case '4':
                    if (b === 0) {
                        alert('No se puede dividir entre 0');
                    } else {
                        alert(a / b);
                    }
                    break;
            }
        } else if (opcion !== '0') {
            alert('Opción no válida');
        }
    } while (opcion !== '0');
}
```

Nivel 4 · arguments, contar en objetos y primos

Objetivo: usar un **objeto como contador o como cajones** (la clave es el nombre del cajón), que es la técnica de agrupar que reaparece en $yedra, el Mercadillo y la Biblioteca.

La técnica del objeto-cajonera

```
let cajones = {};                    // empieza vacío
let clave = 'a';                     // el cajón que toca
if (cajones[clave] === undefined) {  // ¿no existe el cajón?
    cajones[clave] = 0;              //   lo creo
}
cajones[clave]++;                    // meto algo en él
```

Siempre igual: **¿existe? → si no, créalo → úsalo**. Con un contador el cajón empieza en 0 (o 1); con una lista empieza en `[]`.

Ejemplo resuelto 1 · Contar caracteres (Lógica, ej. 10)

Qué piden: recibir un texto y devolver un objeto clave-valor con cuántas veces sale cada carácter.

```
function contarCaracteres(texto) {
    let contador = {};
    for (let i = 0; i < texto.length; i++) {
        let letra = texto[i];
        if (contador[letra] === undefined) {
            contador[letra] = 1;
        } else {
            contador[letra]++;
        }
    }
    return contador;
}
```

```
let resultado = contarCaracteres('banana');   // { b: 1, a: 3, n: 2 }
for (let letra in resultado) {
    console.log(`'${letra}' aparece ${resultado[letra]} veces`);
}
```

Traza con `'banana'`:

i	letra	¿existía?	contador después
0	b	no	`{ b: 1 }`
1	a	no	`{ b: 1, a: 1 }`
2	n	no	`{ b: 1, a: 1, n: 1 }`
3	a	sí	`{ b: 1, a: 2, n: 1 }`
4	n	sí	`{ b: 1, a: 2, n: 2 }`
5	a	sí	`{ b: 1, a: 3, n: 2 }`

Por qué `contador[letra]` y no `contador.letra`: con punto buscaría una clave que se llama literalmente "letra". Con corchetes usa **lo que vale** la variable.

Ejemplo resuelto 2 · Agrupar parámetros por tipo (Lógica, ej. 9)

Qué piden: una función con un número variable de parámetros que los agrupe por tipo y muestre, por cada tipo, los valores con su posición original.

Piensa: número variable de parámetros → `arguments`. La clave del cajón es `typeof arguments[i]`. Cada cajón es una **lista** de objetos `{ posicion, valor }`.

```
function agruparPorTipo() {
    let grupos = {};
    for (let i = 0; i < arguments.length; i++) {
        let tipo = typeof arguments[i];
        if (grupos[tipo] === undefined) {
            grupos[tipo] = [];
        }
        grupos[tipo].push({ posicion: i, valor: arguments[i] });
    }
    return grupos;
}

function mostrarGrupos(grupos) {
    let texto = '';
    for (let tipo in grupos) {
        texto += tipo + ': ';
        for (let elemento of grupos[tipo]) {
            texto += `[${elemento.posicion}] ${elemento.valor}  `;
        }
        texto += '\n';
    }
    console.log(texto);
}

mostrarGrupos(agruparPorTipo(1, 'hola', true, 5, 'adiós', null, undefined));
```

Salida:

```
number: [0] 1  [3] 5
string: [1] hola  [4] adiós
boolean: [2] true
object: [5] null
undefined: [6] undefined
```

Fíjate en `null` → tipo `object` (la trampa de T2). Separamos la lógica (`agruparPorTipo` devuelve datos) de la presentación (`mostrarGrupos` los enseña): así lo pide el profesor.

Ejemplo resuelto 3 · Múltiplos y primos en un rango (Lógica, ej. 11)

Qué piden: para cada número del rango, decir si es múltiplo de 3, de 5 y si es primo. Solo mostrar los que cumplan algo.

Piensa: primo = mayor que 1 y sin divisores entre 2 y n-1. En cuanto encuentro un divisor, ya sé que **no** es primo (`return false`). Si el bucle termina sin encontrar ninguno, **sí** lo es (`return true` después del bucle). Es la misma lógica que `estaDisponible` en la Biblioteca.

```
function esPrimo(n) {
    if (n < 2) {
        return false;
    }
    for (let divisor = 2; divisor < n; divisor++) {
        if (n % divisor === 0) {
            return false;     // encontrado un divisor: no es primo
        }
    }
    return true;              // ninguno lo dividió
}

function analizarRango(inicio, fin) {
    for (let n = inicio; n <= fin; n++) {
        let texto = '';
        if (n % 3 === 0) {
            texto += ' múltiplo de 3';
        }
        if (n % 5 === 0) {
            texto += ' múltiplo de 5';
        }
        if (esPrimo(n)) {
            texto += ' primo';
        }
        if (texto !== '') {
            console.log(n + ':' + texto);
        }
    }
}
analizarRango(1, 16);
// 2: primo · 3: múltiplo de 3 primo · 5: múltiplo de 5 primo · 6: múltiplo de 3 · 7: primo
// 9: múltiplo de 3 · 10: múltiplo de 5 · 11: primo · 12: múltiplo de 3 · 13: primo
// 15: múltiplo de 3 múltiplo de 5
```

Tres `if` **separados** (no `else if`), porque un número puede cumplir varias cosas a la vez (15 es múltiplo de 3 y de 5).

Ejercicio propuesto 4 · Contar palabras por longitud

Recibe un array de palabras y devuelve un objeto donde la clave es la longitud y el valor la lista de palabras con esa longitud. `agruparPorLongitud(['sol', 'mar', 'casa'])` → `{ 3: ['sol', 'mar'], 4: ['casa'] }`.

Solución:

```
function agruparPorLongitud(palabras) {
    let grupos = {};
    for (let palabra of palabras) {
        let largo = palabra.length;
        if (grupos[largo] === undefined) {
            grupos[largo] = [];
        }
        grupos[largo].push(palabra);
    }
    return grupos;
}
```

Nivel 5 · Juegos con números aleatorios

Objetivo: generar aleatorios, controlar intentos y guardar un **historial en un array de objetos**.

Función que reutilizarás en todos los juegos (fórmula de T7):

```
function aleatorio(min, max) {
    return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

Ejemplo resuelto 1 · Adivinar el número (Lógica, ej. 12)

Qué piden: número secreto de 1 a 100, 5 intentos, decir si es mayor o menor, ganar si acierta y enseñar el secreto si pierde.

Piensa el bucle: sigo mientras **queden intentos Y no haya acertado**. Como puede que no haya que preguntar nada más, uso `while`. Un número fuera de rango no gasta intento.

```
function adivinarNumero() {
    let secreto = aleatorio(1, 100);
    let intentos = 5;
    let acertado = false;
    while (intentos > 0 && !acertado) {
        let numero = parseInt(prompt(`Número entre 1 y 100 (te quedan ${intentos}
intentos)`));
        if (isNaN(numero) || numero < 1 || numero > 100) {
            alert('Número fuera de rango');
        } else {
            intentos--;
            if (numero === secreto) {
                acertado = true;
            } else if (numero < secreto) {
                alert('El número secreto es MAYOR');
            } else {
```

```
                alert('El número secreto es MENOR');
            }
        }
    }
    if (acertado) {
        alert('¡Has ganado!');
    } else {
        alert(`Has perdido. El número era ${secreto}`);
    }
}
```

Prueba con secreto 43: `50` → "MENOR" (quedan 4); `abc` → "fuera de rango" (siguen 4); `20` → "MAYOR" (quedan 3); `43` → gana. **Truco para depurar:** pon `console.log(secreto)` al principio para saber el número mientras pruebas, y quítalo al acabar.

La variable `acertado` es una **bandera**: un booleano que cambia cuando pasa algo y que el `while` consulta. La verás en el Simon (`mostrandoSecuencia`) y en los menús.

Ejemplo resuelto 2 · Juego de mates (Lógica, ej. 13)

Qué piden: dos aleatorios de 1 a 10 y un operador aleatorio; el usuario responde; guardar pregunta, respuesta y acierto **ordenadamente en un array**; preguntas en lotes de 4; al terminar, resumen y totales.

Piensa los datos primero:

- Cada pregunta → un objeto `{ texto: '2 / 3', solucion: 0.66 }`.
- El historial → un array de objetos `{ pregunta, respuesta, acierto }`.
- El operador al azar → un array `['+', '-', '*', '/']` y un índice aleatorio de 0 a 3.
- Lotes de 4 → un `for` de 4 dentro de un `do...while` que pregunta si sigue.

```
function generarPregunta() {
    let operadores = ['+', '-', '*', '/'];
    let a = aleatorio(1, 10);
    let b = aleatorio(1, 10);
    let operador = operadores[aleatorio(0, 3)];
    let solucion;
    switch (operador) {
        case '+':
            solucion = a + b;
            break;
        case '-':
            solucion = a - b;
            break;
        case '*':
            solucion = a * b;
            break;
        case '/':
            solucion = Math.trunc((a / b) * 100) / 100; // 2 decimales truncados
            break;
    }
    return { texto: `${a} ${operador} ${b}`, solucion: solucion };
}

function juegoMates() {
    let historial = [];
    let seguir;
```

```
    do {
        for (let i = 0; i < 4; i++) {
            let pregunta = generarPregunta();
            let respuesta = parseFloat(prompt(`¿Cuánto es ${pregunta.texto}? (divisiones con
2 decimales truncados)`));
            let acierto = respuesta === pregunta.solucion;
            alert(acierto ? 'Correcto' : `Incorrecto, era ${pregunta.solucion}`);
            historial.push({ pregunta: pregunta.texto, respuesta: respuesta, acierto: acierto
});
        }
        seguir = prompt('¿Otro lote de 4? (s/n)');
    } while (seguir === 's');

    let resumen = '';
    let aciertos = 0;
    for (let i = 0; i < historial.length; i++) {
        let h = historial[i];
        resumen += `${i + 1}. ${h.pregunta} = ${h.respuesta} → ${h.acierto ? 'acierto' :
'fallo'}\n`;
        if (h.acierto) {
            aciertos++;
        }
    }
    resumen += `Aciertos: ${aciertos}  Fallos: ${historial.length - aciertos}`;
    alert(resumen);
}
```

Salida real de una partida:

```
1. 1 + 6 = 7 → acierto
2. 4 - 4 = 0 → acierto
3. 10 * 10 = 0 → fallo
4. 2 / 3 = 1 → fallo
Aciertos: 2  Fallos: 2
```

`let acierto = respuesta === pregunta.solucion;` guarda directamente el `true`/`false` de la comparación: no hace falta un `if`.

Ejercicio propuesto 5 · Piedra, papel o tijera

Partida al mejor de 5 contra la máquina. Guarda cada ronda en un historial `{ jugador, maquina, ganador }` y al final muestra el resumen.

Solución:

```
function ganadorRonda(j, m) {
    if (j === m) {
        return 'empate';
    }
    if ((j === 'piedra' && m === 'tijera') ||
        (j === 'papel' && m === 'piedra') ||
        (j === 'tijera' && m === 'papel')) {
        return 'jugador';
    }
    return 'maquina';
}

function piedraPapelTijera() {
```

```
    let opciones = ['piedra', 'papel', 'tijera'];
    let historial = [];
    for (let ronda = 1; ronda <= 5; ronda++) {
        let jugador;
        do {
            jugador = prompt(`Ronda ${ronda}: piedra, papel o tijera`);
        } while (jugador !== 'piedra' && jugador !== 'papel' && jugador !== 'tijera');
        let maquina = opciones[aleatorio(0, 2)];
        let ganador = ganadorRonda(jugador, maquina);
        historial.push({ jugador: jugador, maquina: maquina, ganador: ganador });
        alert(`Máquina: ${maquina}. Gana: ${ganador}`);
    }
    let texto = '';
    for (let i in historial) {
        texto += `${Number(i) + 1}. ${historial[i].jugador} vs ${historial[i].maquina}:
${historial[i].ganador}\n`;
    }
    alert(texto);
}
```

Nivel 6 · Algoritmos del Refuerzo (sin métodos)

Objetivo: resolver problemas solo con bucles, `if` y arrays, como exige el Refuerzo 1 ("no debes emplear métodos incluidos en los objetos"). Sin `push` añadimos con `lista[lista.length] = valor`; sin `pop` quitamos el último con `lista.length = lista.length - 1`.

Para cada uno: primero la **idea en una frase**, luego el código.

1 · Anagramas

Idea: cuento las letras de la primera palabra (técnica del nivel 4) y las voy "gastando" con la segunda. Si alguna letra no está o se agota, no son anagramas. Si son iguales o de distinta longitud, tampoco.

```
function sonAnagramas(palabra1, palabra2) {
    if (palabra1 === palabra2 || palabra1.length !== palabra2.length) {
        return false;
    }
    let contador = {};
    for (let i = 0; i < palabra1.length; i++) {
        let letra = palabra1[i];
        if (contador[letra] === undefined) {
            contador[letra] = 0;
        }
        contador[letra]++;
    }
    for (let i = 0; i < palabra2.length; i++) {
        let letra = palabra2[i];
        if (contador[letra] === undefined || contador[letra] === 0) {
            return false;
        }
        contador[letra]--;
    }
    return true;
}
// sonAnagramas('amor', 'roma') → true · ('amor', 'amor') → false · ('aab', 'abb') → false
```

2 · Fibonacci

Idea: las posiciones 0 y 1 valen 0 y 1; cada siguiente es la suma de las dos anteriores.

```
function fibonacci(n) {
    let serie = [];
    for (let i = 0; i < n; i++) {
        if (i === 0) {
            serie[i] = 0;
        } else if (i === 1) {
            serie[i] = 1;
        } else {
            serie[i] = serie[i - 1] + serie[i - 2];
        }
    }
    return serie;
}
// fibonacci(9) → [0, 1, 1, 2, 3, 5, 8, 13, 21]
```

3 · Segundo más grande

Idea: recorro una vez llevando dos variables, `mayor` y `segundo`. Si llega uno mayor que `mayor`, el antiguo mayor baja a segundo. Si está entre los dos, pasa a ser segundo. Los repetidos del mayor no cuentan.

```
function segundoMayor(numeros) {
    let mayor;
    let segundo;
    for (let i = 0; i < numeros.length; i++) {
        let n = numeros[i];
        if (mayor === undefined || n > mayor) {
            segundo = mayor;      // el que era mayor baja
            mayor = n;
        } else if (n < mayor && (segundo === undefined || n > segundo)) {
            segundo = n;
        }
    }
    return segundo;
}
// segundoMayor([4, 9, 2, 9, 7]) → 7 · segundoMayor([5, 5]) → undefined
```

n	mayor	segundo
4	4	undefined
9	9	4
2	9	4
9	9	4 (repetido, no cuenta)
7	9	7

4 · Números perdidos

Idea: busco mínimo y máximo; recorro del mínimo al máximo y, para cada número, miro si está en el array. Si no está, lo apunto.

```
function numerosPerdidos(numeros) {
    let minimo = numeros[0];
    let maximo = numeros[0];
    for (let i = 1; i < numeros.length; i++) {
        if (numeros[i] < minimo) minimo = numeros[i];
        if (numeros[i] > maximo) maximo = numeros[i];
    }
    let perdidos = [];
    for (let n = minimo; n <= maximo; n++) {
        let esta = false;
        for (let i = 0; i < numeros.length; i++) {
            if (numeros[i] === n) {
                esta = true;
            }
        }
        if (!esta) {
            perdidos[perdidos.length] = n;
        }
    }
    return perdidos;
}
// numerosPerdidos([7, 3, 5, 10]) → [4, 6, 8, 9]
```

5 · Conjuntos sin Set

Idea: una función `contiene(lista, valor)` y tres recorridos. Unión: todo lo de A y B sin repetir. Intersección: lo de A que también está en B. Diferencia: lo de A que **no** está en B.

```
function contiene(lista, valor) {
    for (let i = 0; i < lista.length; i++) {
        if (lista[i] === valor) {
            return true;
        }
    }
    return false;
}

function conjuntos(a, b, operacion) {
    let resultado = [];
    switch (operacion) {
        case 'union':
            for (let i = 0; i < a.length; i++) {
                if (!contiene(resultado, a[i])) resultado[resultado.length] = a[i];
            }
            for (let i = 0; i < b.length; i++) {
                if (!contiene(resultado, b[i])) resultado[resultado.length] = b[i];
            }
            break;
        case 'interseccion':
            for (let i = 0; i < a.length; i++) {
                if (contiene(b, a[i]) && !contiene(resultado, a[i]))
resultado[resultado.length] = a[i];
            }
            break;
        case 'diferencia':
            for (let i = 0; i < a.length; i++) {
                if (!contiene(b, a[i]) && !contiene(resultado, a[i]))
resultado[resultado.length] = a[i];
```

```
            }
            break;
    }
    return resultado;
}
// A = [1, 2, 2, 3, 4], B = [3, 4, 5] → unión [1, 2, 3, 4, 5] · intersección [3, 4] ·
diferencia [1, 2]
```

6 · Máquina de cambio

Idea: recorro los valores de mayor a menor. Cuántos caben: `Math.floor(importe / valor)`. Lo que queda: `importe % valor`.

```
function calcularCambio(importe) {
    let valores = [200, 100, 50, 10, 5, 2, 1];
    let cambio = [];
    for (let i = 0; i < valores.length; i++) {
        let cantidad = Math.floor(importe / valores[i]);
        if (cantidad > 0) {
            cambio[cambio.length] = { valor: valores[i], cantidad: cantidad };
            importe = importe % valores[i];
        }
    }
    return cambio;
}
```

Con 388: 200×1 (quedan 188) → 100×1 (88) → 50×1 (38) → 10×3 (8) → 5×1 (3) → 2×1 (1) → 1×1 (0). La parte web se hace igual que el esqueleto de T10: dos inputs (importe producto, entregado), un botón Cobrar y dos `div` de salida.

7 · Expresiones equilibradas (la pila)

Idea: una **pila** es un array donde solo añado y quito por el final. Cada apertura `( [ {` la apilo. Cada cierre debe coincidir con la **última** apertura apilada: si coincide, la quito; si no, está mal. Al final, la pila debe quedar vacía.

```
function estaEquilibrada(expresion) {
    let pila = [];
    for (let i = 0; i < expresion.length; i++) {
        let c = expresion[i];
        if (c === '(' || c === '[' || c === '{') {
            pila[pila.length] = c;                          // apilar
        } else if (c === ')' || c === ']' || c === '}') {
            if (pila.length === 0) {
                return false;                               // cierre sin apertura
            }
            let ultimo = pila[pila.length - 1];
            if ((c === ')' && ultimo !== '(') || (c === ']' && ultimo !== '[') || (c === '}'
&& ultimo !== '{')) {
                return false;                               // no empareja
            }
            pila.length = pila.length - 1;                  // desapilar
        }
    }
    return pila.length === 0;
}
```

Expresión `{ [ ( ) ] }`	Pila después
`{`	`{`
`[`	`{ [`
`(`	`{ [ (`
`)` empareja con `(`	`{ [`
`]` empareja con `[`	`{`
`}` empareja con `{`	vacía → equilibrada

Con `{ a * ( c + d ) ] - 5 }`: al llegar `]` la última apertura es `{` → no empareja → `false`.

8 · Cifrado César en una IIFE

Idea: busco la posición de la letra en el alfabeto, le sumo la clave y doy la vuelta con `%`. Descifrar es desplazar con `-clave` (y si queda negativo, sumo la longitud del alfabeto).

```
const $cesar = (function () {
    const ALFABETO = 'ABCDEFGHIJKLMNÑOPQRSTUVWXYZ';

    function desplazar(texto, clave) {
        let resultado = '';
        for (let i = 0; i < texto.length; i++) {
            let letra = texto[i];
            let posicion = -1;
            for (let j = 0; j < ALFABETO.length; j++) {
                if (ALFABETO[j] === letra) {
                    posicion = j;
                }
            }
            if (posicion === -1) {
                resultado += letra;          // espacios y signos se quedan igual
            } else {
                let nueva = (posicion + clave) % ALFABETO.length;
                if (nueva < 0) {
                    nueva += ALFABETO.length;
                }
                resultado += ALFABETO[nueva];
            }
        }
        return resultado;
    }

    function cifrarTexto(textoPlano, clave) {
        return desplazar(textoPlano, clave);
    }

    function descifrarTexto(textoCifrado, clave) {
        return desplazar(textoCifrado, -clave);
    }

    return { cifrarTexto, descifrarTexto };
})();
// $cesar.cifrarTexto('HOLA ZORRO', 3) → 'KRÑD CRUUR' · cifrar 'XYZ' con 3 → 'ABC'
```

`desplazar` es privada: desde fuera solo existen `cifrarTexto` y `descifrarTexto`. La vuelta: Z está en 26; 26 + 3 = 29; 29 % 27 = 2 → C.

9 · Tres en raya: comprobar el estado

Idea: el tablero es una matriz 3×3. Hay **8 líneas** ganadoras (3 filas, 3 columnas, 2 diagonales). Las meto en un array y compruebo si alguna tiene las tres casillas iguales y no vacías. Si no gana nadie y no quedan huecos, empate.

```
function comprobarEstado(m) {
    let lineas = [
        [m[0][0], m[0][1], m[0][2]], [m[1][0], m[1][1], m[1][2]], [m[2][0], m[2][1], m[2]
[2]],
        [m[0][0], m[1][0], m[2][0]], [m[0][1], m[1][1], m[2][1]], [m[0][2], m[1][2], m[2]
[2]],
        [m[0][0], m[1][1], m[2][2]], [m[0][2], m[1][1], m[2][0]]
    ];
    for (let linea of lineas) {
        if (linea[0] !== '' && linea[0] === linea[1] && linea[1] === linea[2]) {
            return linea[0];               // 'x' u 'o'
        }
    }
    for (let fila of m) {
        for (let celda of fila) {
            if (celda === '') {
                return '';                 // quedan huecos: sigue la partida
            }
        }
    }
    return 'empate';
}
```

Con esta función, `ponerMaquina` sigue el orden del enunciado: para cada hueco, prueba a poner su ficha y llama a `comprobarEstado`; si gana, la deja. Si no, prueba con la ficha del rival para bloquearle. Si tampoco, pone en un hueco aleatorio.

Ejercicio propuesto 6 · Palíndromo

Función que diga si una palabra se lee igual al revés (`'reconocer'` → true), sin métodos.

Solución:

```
function esPalindromo(palabra) {
    let i = 0;
    let j = palabra.length - 1;
    while (i < j) {
        if (palabra[i] !== palabra[j]) {
            return false;
        }
        i++;
        j--;
    }
    return true;
}
```

Nivel 7 · Arrays de objetos JSON y CRUD con menú

Objetivo: trabajar con un **array de objetos** como si fuera una pequeña base de datos: listar, crear, buscar, modificar, borrar y calcular. Es el mismo CRUD que harás luego con clases en la Juguetería y la Biblioteca.

Las cuatro operaciones que se repiten siempre

Operación	Cómo	Ejemplo
Listar	recorrer y construir un texto	`for (let t of trabajadores) texto += ...`
Crear	crear el objeto y `push`	`trabajadores.push({ codigo, nombre })`
Buscar	`for...in` comparando el campo; devolver la posición (o -1) o el objeto (o `undefined`)	`if (trabajadores[i].codigo === codigo) return Number(i);`
Borrar	buscar la posición y `splice(posicion, 1)`	`trabajadores.splice(posicion, 1)`

Ejemplo resuelto 1 · Gestión académica (Lógica, ej. 14)

Qué piden: alumnos `{ nombre, asignaturas }` con asignaturas `{ modulo, nota }`. Una función añade a cada alumno `promociona` (todo aprobado) y `media`. Otra lista los que promocionan (`Índice-Nombre-Media`) y otra los que no (`Índice-Nombre-Pendientes:[...]`).

Piensa: un objeto anidado. Para cada alumno (bucle de fuera) recorro sus asignaturas (bucle de dentro) sumando notas y bajando la bandera `promociona` si alguna es menor que 5. Las propiedades nuevas se añaden asignándolas: `alumno.media = ...`.

```
let alumnos = [
    { nombre: 'Ana', asignaturas: [{ modulo: 'DWEC', nota: 7 }, { modulo: 'DIW', nota: 8 }, {
modulo: 'DWES', nota: 6 }, { modulo: 'DAW', nota: 9 }] },
    { nombre: 'Luis', asignaturas: [{ modulo: 'DWEC', nota: 4 }, { modulo: 'DIW', nota: 6 },
{ modulo: 'DWES', nota: 3 }, { modulo: 'DAW', nota: 7 }] },
    { nombre: 'Marta', asignaturas: [{ modulo: 'DWEC', nota: 5 }, { modulo: 'DIW', nota: 5 },
{ modulo: 'DWES', nota: 9 }, { modulo: 'DAW', nota: 6 }] }
];

function calcularExpedientes(alumnos) {
    for (let alumno of alumnos) {
        let suma = 0;
        let promociona = true;
        for (let asignatura of alumno.asignaturas) {
            suma += asignatura.nota;
            if (asignatura.nota < 5) {
                promociona = false;
            }
        }
        alumno.media = suma / alumno.asignaturas.length;   // propiedad nueva
        alumno.promociona = promociona;                     // propiedad nueva
    }
}

function listarPromocionan(alumnos) {
    for (let i in alumnos) {
        if (alumnos[i].promociona) {
```

```
            console.log(`${i}-${alumnos[i].nombre}-${alumnos[i].media}`);
        }
    }
}

function listarNoPromocionan(alumnos) {
    for (let i in alumnos) {
        let alumno = alumnos[i];
        if (!alumno.promociona) {
            let pendientes = [];
            for (let asignatura of alumno.asignaturas) {
                if (asignatura.nota < 5) {
                    pendientes.push(asignatura.modulo);
                }
            }
            console.log(`${i}-${alumno.nombre}-Pendientes:[${pendientes}]`);
        }
    }
}

calcularExpedientes(alumnos);
listarPromocionan(alumnos);    // 0-Ana-7.5 · 2-Marta-6.25
listarNoPromocionan(alumnos);  // 1-Luis-Pendientes:[DWEC,DWES]
```

Un array metido en una plantilla `${pendientes}` se escribe con sus elementos separados por comas.

Ejemplo resuelto 2 · Gestión de personal con menú en una IIFE (Lógica, ej. 15 y 16)

Qué piden: trabajadores `{ codigo, nombre, categoria, contratacion }`. Código automático `E01`, `E02`… que no se puede cambiar. Categorías 1, 2, 3 = 1100, 1400, 1900 €. Nómina = base + 4 % por año de antigüedad. Menú con listar, crear, borrar (con confirmación), modificar (con el valor actual por defecto) y nóminas ordenadas por categoría con subtotales y total. El ejercicio 16 pide envolverlo en una función autoinvocada.

Piensa antes de escribir:

- Datos privados dentro de la IIFE: el array, un `contador` para los códigos y un objeto `SALARIOS` para traducir categoría → euros (`SALARIOS[2]` da 1400).
- "Datos cacheados" = empezar con algunos trabajadores ya cargados para probar.
- Valor por defecto al modificar: el segundo parámetro de `prompt(mensaje, valorPorDefecto)`.
- Nóminas por categoría: ordeno una **copia** (`toSorted`) por categoría y voy sumando; cuando el siguiente es de otra categoría (o no hay siguiente), escribo el subtotal y lo pongo a 0.
- Hacia fuera solo hace falta `menu`.

```
const $personal = (function () {
    const SALARIOS = { 1: 1100, 2: 1400, 3: 1900 };
    let trabajadores = [
        { codigo: 'E01', nombre: 'Ana', categoria: 3, contratacion: 2015 },
        { codigo: 'E02', nombre: 'Luis', categoria: 1, contratacion: 2022 },
        { codigo: 'E03', nombre: 'Marta', categoria: 1, contratacion: 2020 }
    ];
    let contador = 3;
```

```
    function generarCodigo() {
        contador++;
        if (contador < 10) {
            return 'E0' + contador;
        }
        return 'E' + contador;
    }

    function buscarPosicion(codigo) {
        for (let i in trabajadores) {
            if (trabajadores[i].codigo === codigo) {
                return Number(i);
            }
        }
        return -1;
    }

    function pedirCategoria(valorPorDefecto) {
        let categoria;
        do {
            categoria = parseInt(prompt('Categoría (1, 2 o 3)', valorPorDefecto));
        } while (categoria !== 1 && categoria !== 2 && categoria !== 3);
        return categoria;
    }

    function pedirAnio(valorPorDefecto) {
        let anio;
        let actual = new Date().getFullYear();
        do {
            anio = parseInt(prompt('Año de contratación', valorPorDefecto));
        } while (isNaN(anio) || anio < 1950 || anio > actual);
        return anio;
    }

    function listar() {
        let texto = '';
        for (let t of trabajadores) {
            texto += `${t.codigo} - ${t.nombre} - cat. ${t.categoria} - desde
${t.contratacion}\n`;
        }
        alert(texto === '' ? 'No hay trabajadores' : texto);
    }

    function crear() {
        let nombre = prompt('Nombre');
        if (!nombre) {
            alert('Nombre obligatorio');
            return;
        }
        let categoria = pedirCategoria(1);
        let contratacion = pedirAnio(new Date().getFullYear());
        trabajadores.push({ codigo: generarCodigo(), nombre: nombre, categoria: categoria,
contratacion: contratacion });
    }

    function borrar() {
        let posicion = buscarPosicion(prompt('Código del trabajador a borrar'));
        if (posicion === -1) {
            alert('No existe ese trabajador');
            return;
        }
```

```
        let respuesta = prompt(`¿Seguro que quieres borrar a
${trabajadores[posicion].nombre}? (s/n)`);
        if (respuesta === 's') {
            trabajadores.splice(posicion, 1);
            alert('Trabajador borrado');
        }
    }

    function modificar() {
        let posicion = buscarPosicion(prompt('Código del trabajador a modificar'));
        if (posicion === -1) {
            alert('No existe ese trabajador');
            return;
        }
        let t = trabajadores[posicion];
        let nombre = prompt('Nombre', t.nombre);        // sale escrito el nombre actual
        if (nombre) {
            t.nombre = nombre;
        }
        t.categoria = pedirCategoria(t.categoria);
        t.contratacion = pedirAnio(t.contratacion);
        // el código no se toca
    }

    function calcularNomina(trabajador) {
        let antiguedad = new Date().getFullYear() - trabajador.contratacion;
        let base = SALARIOS[trabajador.categoria];
        return base + base * 0.04 * antiguedad;
    }

    function nominas() {
        let ordenados = trabajadores.toSorted((a, b) => a.categoria - b.categoria);
        let texto = '';
        let totalCategoria = 0;
        let totalEmpresa = 0;
        for (let i = 0; i < ordenados.length; i++) {
            let t = ordenados[i];
            let nomina = calcularNomina(t);
            texto += `Cat. ${t.categoria} - ${t.nombre}: ${Math.round(nomina * 100) / 100} €
\n`;
            totalCategoria += nomina;
            totalEmpresa += nomina;
            let esUltimoDeSuCategoria = i === ordenados.length - 1 || ordenados[i +
1].categoria !== t.categoria;
            if (esUltimoDeSuCategoria) {
                texto += `  Total categoría ${t.categoria}: ${Math.round(totalCategoria *
100) / 100} €\n`;
                totalCategoria = 0;
            }
        }
        texto += `TOTAL EMPRESA: ${Math.round(totalEmpresa * 100) / 100} €`;
        alert(texto);
    }

    function menu() {
        let opcion;
        do {
            opcion = prompt('1. Listar\n2. Crear\n3. Borrar\n4. Modificar\n5. Nóminas\n0.
Terminar');
            switch (opcion) {
                case '1':
```

```
                    listar();
                    break;
                case '2':
                    crear();
                    break;
                case '3':
                    borrar();
                    break;
                case '4':
                    modificar();
                    break;
                case '5':
                    nominas();
                    break;
                case '0':
                    break;
                default:
                    alert('Opción no válida');
            }
        } while (opcion !== '0');
    }

    return { menu };
})();

$personal.menu();
```

Salida de las nóminas en 2026 tras crear a Pepe (cat. 3, 2024) y borrar a Luis:

```
Cat. 1 - Marta: 1364 €            1100 + 1100 · 0.04 · 6 años
  Total categoría 1: 1364 €
Cat. 3 - Ana: 2736 €              1900 + 1900 · 0.04 · 11 años
Cat. 3 - Pepe: 2052 €             1900 + 1900 · 0.04 · 2 años
  Total categoría 3: 4788 €
TOTAL EMPRESA: 6152 €
```

Por qué `Number(i)` en `buscarPosicion`: el `i` de `for...in` es texto (`'1'`). Si devuelves texto, `posicion === -1` funciona igual, pero luego operar con él da sorpresas (`'1' + 1` = `'11'`).

Por qué `toSorted` y no `sort`: `sort` reordenaría el array original y el listado normal saldría cambiado.

Ejercicio propuesto 7 · Biblioteca de discos con menú

Discos `{ titulo, grupo, anio }`. Menú: listar, añadir, borrar por título (con confirmación), y listar ordenados por año del más antiguo al más nuevo. Todo dentro de una IIFE `$discos` que solo expone `menu`.

Pista de solución: copia la estructura de `$personal`: `buscarPosicion(titulo)` con `for...in`, `splice` para borrar y `discos.toSorted((a, b) => a.anio - b.anio)` para el listado ordenado.

Nivel 8 · Closures y la IIFE $yedra (UD2E4)

Objetivo: entender que una función puede **fabricar** otra función con datos guardados (closure), y montar un módulo de datos con IIFE que agrupa, calcula medias, ordena y convierte a JSON.

Ejemplo resuelto 1 · El closure jugarDados (UD2E4, ej. 1)

Qué piden: `jugarDados(numeroLados)` define dentro una función que da un aleatorio de 1 a `numeroLados` y devuelve una función anónima que tira **dos** dados y devuelve un array. Después, un script pide lados y tiradas, crea `tirarDados`, compara jugador y máquina y muestra resultados y victorias.

Entiéndelo antes de escribirlo:

```
let tirarDados = jugarDados(6);
                 └─ se ejecuta UNA vez: guarda numeroLados = 6 y
                    devuelve la función anónima (no la ejecuta)

tirarDados();   → [3, 5]   ┐ cada llamada ejecuta la anónima, que
tirarDados();   → [6, 1]   ┘ todavía "recuerda" numeroLados = 6
```

```
function jugarDados(numeroLados) {
    function tirarUnDado() {                        // función interna (privada)
        return Math.floor(Math.random() * numeroLados) + 1;
    }
    return function () {                            // la anónima que se devuelve
        return [tirarUnDado(), tirarUnDado()];
    };
}

function probarDados() {
    let lados = parseInt(prompt('Número de lados de los dados'));
    let tiradas = parseInt(prompt('Número de tiradas'));
    if (isNaN(lados) || lados < 2 || isNaN(tiradas) || tiradas < 1) {
        alert('Datos no válidos');
        return;
    }
    let tirarDados = jugarDados(lados);
    let victoriasJugador = 0;
    let victoriasMaquina = 0;
    for (let i = 1; i <= tiradas; i++) {
        let jugador = tirarDados();
        let maquina = tirarDados();
        let sumaJugador = jugador[0] + jugador[1];
        let sumaMaquina = maquina[0] + maquina[1];
        let ganador;
        if (sumaJugador > sumaMaquina) {
            ganador = 'jugador';
            victoriasJugador++;
        } else if (sumaMaquina > sumaJugador) {
            ganador = 'máquina';
            victoriasMaquina++;
        } else {
            ganador = 'empate';
        }
        alert(`Tirada ${i}: jugador ${jugador[0]}+${jugador[1]}=${sumaJugador}, máquina
${maquina[0]}+${maquina[1]}=${sumaMaquina} → ${ganador}`);
    }
    alert(`Victorias jugador: ${victoriasJugador}\nVictorias máquina: ${victoriasMaquina}`);
}
```

Salida de una prueba con 6 lados y 2 tiradas: `Tirada 1: jugador 6+1=7, máquina 2+2=4 → jugador`, `Tirada 2: jugador 4+4=8, máquina 4+4=8 → empate`, `Victorias jugador: 1, máquina: 0`.

La versión con página HTML está en el nivel 9.

Ejemplo resuelto 2 · $yedra: JSON y arrays (UD2E4, ej. 2)

Qué piden: alumnos `{ nombre, nota, modulo, convocatorias }` (nota 0-10, convocatorias 1-4, módulos DWEC y DWES). Cuatro funciones dentro de una IIFE `$yedra`:

1. Suspensos (nota < 5) con solo nombre, nota y módulo, ordenados por nombre.
2. Estadísticas por módulo: nota media y convocatorias medias, truncadas a 2 decimales, ordenadas por convocatorias de mayor a menor. **Primero agrupar** en un objeto y **luego** calcular.
3. Devolver todos los datos como cadena JSON.
4. Cargar una cadena JSON nueva con `try...catch` y comprobar que es un array.

Dibuja los datos de la función 2 (es la técnica de cajones del nivel 4 con un objeto por cajón):

```
alumnos ──agrupar──▶ grupos = {
                       DWEC: { sumaNotas: 21, sumaConvocatorias: 5, cantidad: 3 },
                       DWES: { sumaNotas: 11, sumaConvocatorias: 7, cantidad: 3 }
                     }
        ──calcular──▶ [ { modulo: 'DWES', notaMedia: 3.66, convocatoriasMedia: 2.33 },
                       { modulo: 'DWEC', notaMedia: 7,    convocatoriasMedia: 1.66 } ]
```

```
const $yedra = (function () {
    let alumnos = [
        { nombre: 'Lucía', nota: 8, modulo: 'DWEC', convocatorias: 1 },
        { nombre: 'Andrés', nota: 3, modulo: 'DWES', convocatorias: 2 },
        { nombre: 'Carla', nota: 4, modulo: 'DWEC', convocatorias: 3 },
        { nombre: 'Bruno', nota: 6, modulo: 'DWES', convocatorias: 1 },
        { nombre: 'Elena', nota: 9, modulo: 'DWEC', convocatorias: 1 },
        { nombre: 'Diego', nota: 2, modulo: 'DWES', convocatorias: 4 }
    ];

    function listarSuspensos() {
        let suspensos = [];
        for (let i in alumnos) {
            let alumno = alumnos[i];
            if (alumno.nota < 5) {
                suspensos.push({ nombre: alumno.nombre, nota: alumno.nota, modulo:
alumno.modulo });
            }
        }
        suspensos.sort((a, b) => a.nombre.localeCompare(b.nombre));
        return suspensos;
    }

    function estadisticasPorModulo() {
        // 1. agrupar
        let grupos = {};
        for (let i in alumnos) {
            let alumno = alumnos[i];
            if (grupos[alumno.modulo] === undefined) {
                grupos[alumno.modulo] = { sumaNotas: 0, sumaConvocatorias: 0, cantidad: 0 };
            }
            grupos[alumno.modulo].sumaNotas += alumno.nota;
```

```
            grupos[alumno.modulo].sumaConvocatorias += alumno.convocatorias;
            grupos[alumno.modulo].cantidad++;
        }
        // 2. calcular medias
        let resultado = [];
        for (let modulo in grupos) {
            let g = grupos[modulo];
            resultado.push({
                modulo: modulo,
                notaMedia: Math.trunc((g.sumaNotas / g.cantidad) * 100) / 100,
                convocatoriasMedia: Math.trunc((g.sumaConvocatorias / g.cantidad) * 100) /
100
            });
        }
        // 3. ordenar de mayor a menor convocatorias
        resultado.sort((a, b) => b.convocatoriasMedia - a.convocatoriasMedia);
        return resultado;
    }

    function obtenerJSON() {
        return JSON.stringify(alumnos);
    }

    function cargarJSON(cadena) {
        try {
            let datos = JSON.parse(cadena);
            if (!Array.isArray(datos)) {
                alert('El JSON no contiene un array');
                return;
            }
            alumnos = datos;
            alert('Datos cargados');
        } catch (error) {
            alert('JSON no válido: ' + error.message);
        }
    }

    return { listarSuspensos, estadisticasPorModulo, obtenerJSON, cargarJSON };
})();
```

Resultados comprobados:

Llamada	Resultado
`$yedra.listarSuspensos()`	Andrés 3 DWES, Carla 4 DWEC, Diego 2 DWES
`$yedra.estadisticasPorModulo()`	DWES 3.66 / 2.33, luego DWEC 7 / 1.66
`$yedra.cargarJSON('{"nombre":Juan}')`	alert: JSON no válido
`$yedra.cargarJSON('{"nombre":"Juan"}')`	alert: no contiene un array
`$yedra.cargarJSON('[{"nombre":"Zoe",...}]')`	alert: Datos cargados

Por qué `alumnos` es `let` y no `const`: `cargarJSON` lo **reasigna** con el array nuevo. Y por qué las funciones siguen viendo el array nuevo: todas leen la misma variable privada de la IIFE (es un closure).

Ejercicio propuesto 8 · Contador de visitas con closure

Crea `crearContadorVisitas(pagina)` que devuelva una función; cada vez que se llama suma una visita y devuelve `'inicio: 3 visitas'`. Dos contadores distintos no se mezclan.

Solución:

```
function crearContadorVisitas(pagina) {
    let visitas = 0;
    return function () {
        visitas++;
        return `${pagina}: ${visitas} visitas`;
    };
}
let inicio = crearContadorVisitas('inicio');
let contacto = crearContadorVisitas('contacto');
inicio();    // 'inicio: 1 visitas'
inicio();    // 'inicio: 2 visitas'
contacto();  // 'contacto: 1 visitas'
```

Nivel 9 · Primeras páginas con DOM: Dados y Mercadillo

Objetivo: pasar de `prompt` / `alert` a una página con campos, botones y una zona de resultados. La lógica no cambia: solo cambia **de dónde salen los datos** (`input.value`) y **dónde se muestran** (`innerHTML`).

Antes (consola)	Ahora (página)
`prompt('Lados')`	`document.getElementById('lados').value`
`alert(texto)`	`document.getElementById('resultado').innerHTML = texto`
un menú `do...while`	un botón por opción + `addEventListener('click', ...)`
el programa corre de arriba abajo	el programa **espera** a que el usuario pulse

Ejemplo resuelto 1 · Partida de dados con botón (UD2E4, ej. 1, parte web)

Qué piden: página para jugar contra la máquina: lados, número de tiradas, resultados de cada tirada y totales. El jugador pulsa un botón cada vez que quiere tirar.

Piensa el estado: antes, todo pasaba dentro de un `for`. Ahora cada tirada es un clic, así que lo que debe **sobrevivir entre clics** (la función de dados, tirada actual, victorias) va en una variable fuera de las funciones: `partida`. Empieza en `null` = "no hay partida", y así se puede comprobar con `if (partida === null)`.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Dados</title>
    <script src="dados.js" defer></script>
</head>
<body>
    <div>
        <div>
```

```
            <label>Lados del dado:</label>
            <input type="number" id="lados" value="6">
        </div>
        <div>
            <label>Número de tiradas:</label>
            <input type="number" id="tiradas" value="3">
        </div>
        <div>
            <input type="button" id="btnNueva" value="Nueva partida">
            <input type="button" id="btnTirar" value="Tirar">
        </div>
    </div>
    <div id="tiradasHechas"></div>
    <div id="totales"></div>
</body>
</html>
```

```
'use strict';

function jugarDados(numeroLados) {
    function tirarUnDado() {
        return Math.floor(Math.random() * numeroLados) + 1;
    }
    return function () {
        return [tirarUnDado(), tirarUnDado()];
    };
}

let partida = null;   // null = no hay partida empezada

function nuevaPartida() {
    let lados = parseInt(document.getElementById('lados').value);
    let tiradas = parseInt(document.getElementById('tiradas').value);
    if (isNaN(lados) || lados < 2 || isNaN(tiradas) || tiradas < 1) {
        alert('Lados (mínimo 2) y tiradas (mínimo 1) no válidos');
        return;
    }
    partida = {
        tirarDados: jugarDados(lados),   // el closure se guarda dentro del objeto
        tiradasTotales: tiradas,
        tiradaActual: 0,
        victoriasJugador: 0,
        victoriasMaquina: 0
    };
    document.getElementById('tiradasHechas').innerHTML = '';
    document.getElementById('totales').innerHTML = 'Jugador 0 - Máquina 0';
}

function tirar() {
    if (partida === null) {
        alert('Empieza una nueva partida');
        return;
    }
    if (partida.tiradaActual === partida.tiradasTotales) {
        alert('La partida ha terminado');
        return;
    }
    partida.tiradaActual++;
    let jugador = partida.tirarDados();
    let maquina = partida.tirarDados();
```

```
    let sumaJugador = jugador[0] + jugador[1];
    let sumaMaquina = maquina[0] + maquina[1];
    let ganador = 'empate';
    if (sumaJugador > sumaMaquina) {
        ganador = 'jugador';
        partida.victoriasJugador++;
    } else if (sumaMaquina > sumaJugador) {
        ganador = 'máquina';
        partida.victoriasMaquina++;
    }
    document.getElementById('tiradasHechas').innerHTML +=
        `<div>Tirada ${partida.tiradaActual}: tú ${jugador[0]} + ${jugador[1]} =
${sumaJugador} · máquina ${maquina[0]} + ${maquina[1]} = ${sumaMaquina} → ${ganador}</div>`;
    document.getElementById('totales').innerHTML =
        `Jugador ${partida.victoriasJugador} - Máquina ${partida.victoriasMaquina}`;
    if (partida.tiradaActual === partida.tiradasTotales) {
        alert('Fin de la partida');
    }
}

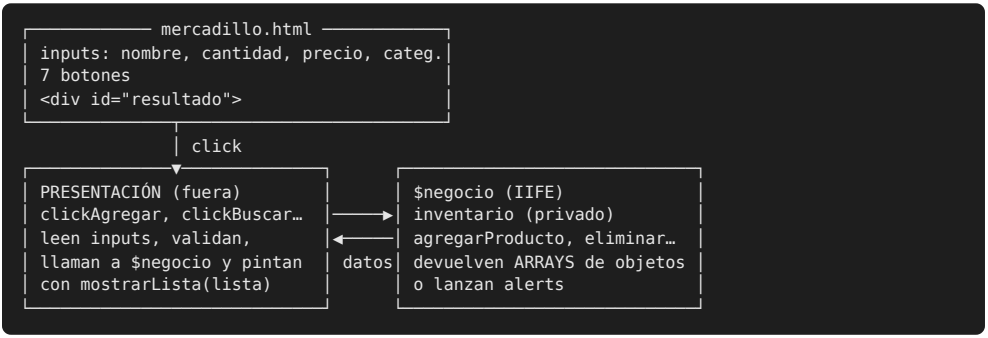
window.addEventListener('load', function () {
    document.getElementById('btnNueva').addEventListener('click', nuevaPartida);
    document.getElementById('btnTirar').addEventListener('click', tirar);
});
```

`innerHTML +=` **añade** una fila nueva sin borrar las anteriores; `innerHTML =` **sustituye** todo. Probado en el navegador: tirar sin partida avisa, 3 tiradas pintan 3 filas, y una cuarta avisa de que ha terminado.

Ejemplo resuelto 2 · Mercadillo completo (UD2E2)

Qué piden: inventario en un objeto cuya clave es el nombre del producto y el valor `{ cantidad, precio, categoria }`. Siete funciones dentro de la IIFE `$negocio` (que solo devuelve datos y lanza alerts). Los eventos se asocian en `load`, **fuera** de `$negocio`, y la presentación es independiente. Confirmación antes de eliminar.

Dibuja la arquitectura antes de programar:



El detalle que avisa el enunciado ("cuidado que no tienes el nombre"): el nombre es la **clave**, no está dentro del valor. Para ordenar o filtrar hay que pasar el objeto a un array **metiendo el nombre** en cada elemento:

```
{ manzana: { cantidad: 10, precio: 1.5, ... } }
        ──for...in──▶ [ { nombre: 'manzana', cantidad: 10, precio: 1.5, ... } ]
```

Eliminar sin `delete`: construyo un inventario nuevo copiando todas las claves menos la que quiero quitar y lo asigno al antiguo.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Mercadillo</title>
    <script src="mercadillo.js" defer></script>
</head>
<body>
    <div>
        <div>
            <label>Nombre:</label>
            <input type="text" id="nombre">
        </div>
        <div>
            <label>Cantidad:</label>
            <input type="number" id="cantidad">
        </div>
        <div>
            <label>Precio:</label>
            <input type="number" id="precio">
        </div>
        <div>
            <label>Categoría:</label>
            <input type="text" id="categoria">
        </div>
        <div>
            <input type="button" id="btnAgregar" value="Agregar">
            <input type="button" id="btnEliminar" value="Eliminar">
            <input type="button" id="btnBuscar" value="Buscar">
            <input type="button" id="btnActualizar" value="Actualizar stock">
            <input type="button" id="btnOrdenar" value="Ordenar por precio">
            <input type="button" id="btnInventario" value="Inventario">
            <input type="button" id="btnFiltrar" value="Filtrar por categoría">
        </div>
    </div>
    <div id="resultado"></div>
</body>
</html>
```

```
'use strict';

// ================== LÓGICA: $negocio ==================
const $negocio = (function () {
    // clave: nombre del producto · valor: { cantidad, precio, categoria }
    let inventario = {
        manzana: { cantidad: 10, precio: 1.5, categoria: 'fruta' },
        pera: { cantidad: 4, precio: 2, categoria: 'fruta' },
        leche: { cantidad: 6, precio: 0.9, categoria: 'lácteo' }
    };

    function agregarProducto(nombre, cantidad, precio, categoria) {
        if (inventario[nombre] !== undefined) {
```

```
            alert(`El producto ${nombre} ya existe`);
            return;
        }
        inventario[nombre] = { cantidad: cantidad, precio: precio, categoria: categoria };
    }

    function eliminarProducto(nombre) {
        if (inventario[nombre] === undefined) {
            alert(`El producto ${nombre} no existe`);
            return;
        }
        let nuevo = {};                          // todos menos ese
        for (let clave in inventario) {
            if (clave !== nombre) {
                nuevo[clave] = inventario[clave];
            }
        }
        inventario = nuevo;
    }

    function buscarProducto(nombre) {
        return inventario[nombre];               // undefined si no existe
    }

    function actualizarInventario(nombre, cantidad) {
        let producto = inventario[nombre];
        if (producto === undefined) {
            alert(`El producto ${nombre} no existe`);
            return;
        }
        if (producto.cantidad + cantidad < 0) {
            alert('No hay stock suficiente');
            return;
        }
        producto.cantidad += cantidad;
        if (producto.cantidad === 0) {
            alert(`Stock de ${nombre} a 0: hay que reponer`);
        }
    }

    function ordenarProductosPorPrecio() {
        let lista = [];
        for (let nombre in inventario) {
            let p = inventario[nombre];
            lista.push({ nombre: nombre, cantidad: p.cantidad, precio: p.precio, categoria:
p.categoria });
        }
        return lista.toSorted((a, b) => a.precio - b.precio);
    }

    function imprimirInventario() {
        let lista = [];
        for (let nombre in inventario) {
            let p = inventario[nombre];
            lista.push({ nombre: nombre, categoria: p.categoria, cantidad: p.cantidad,
precio: p.precio, total: p.cantidad * p.precio });
        }
        return lista;
    }

    function filtrarProductosPorCategoria(categoria) {
```

```
        let lista = [];
        for (let nombre in inventario) {
            let p = inventario[nombre];
            lista.push({ nombre: nombre, cantidad: p.cantidad, precio: p.precio, categoria:
p.categoria });
        }
        return lista
            .filter((p) => p.categoria === categoria)
            .map((p) => {
                return { nombre: p.nombre, cantidad: p.cantidad, precio: p.precio };
            });
    }

    return {
        agregarProducto,
        eliminarProducto,
        buscarProducto,
        actualizarInventario,
        ordenarProductosPorPrecio,
        imprimirInventario,
        filtrarProductosPorCategoria
    };
})();

// ================== PRESENTACIÓN ==================

// Pinta cualquier lista de objetos: cabecera con las claves del primero y una fila por
objeto
function mostrarLista(lista) {
    let resultado = document.getElementById('resultado');
    if (lista.length === 0) {
        resultado.innerHTML = '<div>No hay productos</div>';
        return;
    }
    let html = '<div>';
    for (let clave in lista[0]) {
        html += `<div>${clave}</div>`;
    }
    html += '</div>';
    for (let fila of lista) {
        html += '<div>';
        for (let clave in fila) {
            html += `<div>${fila[clave]}</div>`;
        }
        html += '</div>';
    }
    resultado.innerHTML = html;
}

function clickAgregar() {
    let nombre = document.getElementById('nombre').value.trim();
    let cantidad = parseInt(document.getElementById('cantidad').value);
    let precio = parseFloat(document.getElementById('precio').value);
    let categoria = document.getElementById('categoria').value.trim();
    if (nombre === '' || categoria === '' || isNaN(cantidad) || isNaN(precio)) {
        alert('Rellena todos los campos');
        return;
    }
    $negocio.agregarProducto(nombre, cantidad, precio, categoria);
    mostrarLista($negocio.imprimirInventario());
}
```

```
function clickEliminar() {
    let nombre = document.getElementById('nombre').value.trim();
    let respuesta = prompt(`¿Seguro que quieres eliminar ${nombre}? (s/n)`);
    if (respuesta === 's') {
        $negocio.eliminarProducto(nombre);
        mostrarLista($negocio.imprimirInventario());
    }
}

function clickBuscar() {
    let nombre = document.getElementById('nombre').value.trim();
    let producto = $negocio.buscarProducto(nombre);
    if (!producto) {
        alert('No existe ese producto');
        return;
    }
    mostrarLista([{ nombre: nombre, cantidad: producto.cantidad, precio: producto.precio,
categoria: producto.categoria }]);
}

function clickActualizar() {
    let cantidad = parseInt(document.getElementById('cantidad').value);
    if (isNaN(cantidad)) {
        alert('Escribe la cantidad a sumar (o restar, en negativo)');
        return;
    }
    $negocio.actualizarInventario(document.getElementById('nombre').value.trim(), cantidad);
    mostrarLista($negocio.imprimirInventario());
}

function clickOrdenar() {
    mostrarLista($negocio.ordenarProductosPorPrecio());
}

function clickInventario() {
    mostrarLista($negocio.imprimirInventario());
}

function clickFiltrar() {

mostrarLista($negocio.filtrarProductosPorCategoria(document.getElementById('categoria').value
.trim()));
}

window.addEventListener('load', function () {
    document.getElementById('btnAgregar').addEventListener('click', clickAgregar);
    document.getElementById('btnEliminar').addEventListener('click', clickEliminar);
    document.getElementById('btnBuscar').addEventListener('click', clickBuscar);
    document.getElementById('btnActualizar').addEventListener('click', clickActualizar);
    document.getElementById('btnOrdenar').addEventListener('click', clickOrdenar);
    document.getElementById('btnInventario').addEventListener('click', clickInventario);
    document.getElementById('btnFiltrar').addEventListener('click', clickFiltrar);
    mostrarLista($negocio.imprimirInventario());
});
```

Prueba realizada en el navegador: agregar queso (ok) y otra vez queso (alert "ya existe"); restar 4 a pera (alert "hay que reponer"); ordenar (leche 0.9, manzana 1.5, pera 2, queso 5);

filtrar lácteo (leche y queso); eliminar leche contestando `s`; buscar un producto inexistente (alert). Sin errores en consola.

Cómo construirlo tú, paso a paso (prueba cada paso en la consola con `$negocio.agregarProducto('x', 1, 2, 'y')`, `$negocio.imprimirInventario()`...):

1. El HTML con sus ids.
2. `$negocio` con el inventario de prueba y `imprimirInventario`.
3. `mostrarLista` + el `load` que pinta el inventario al abrir.
4. Una función de `$negocio` y su `click...` cada vez: agregar, buscar, eliminar, actualizar, ordenar, filtrar.

Ejercicio propuesto 9 · Cifrado César en la web (Refuerzo, ej. 14)

Página con un campo para la clave, dos `textarea` (texto plano y cifrado) y dos botones (Cifrar y Descifrar) que usen `$cesar` del nivel 6.

Solución (solo lo nuevo):

```
<div>
    <label>Clave:</label>
    <input type="number" id="clave" value="3">
</div>
<div>
    <label>Texto plano:</label>
    <textarea id="plano"></textarea>
</div>
<div>
    <label>Texto cifrado:</label>
    <textarea id="cifrado"></textarea>
</div>
<div>
    <input type="button" id="btnCifrar" value="Cifrar">
    <input type="button" id="btnDescifrar" value="Descifrar">
</div>
```

```
window.addEventListener('load', function () {
    document.getElementById('btnCifrar').addEventListener('click', function () {
        let clave = parseInt(document.getElementById('clave').value);
        let plano = document.getElementById('plano').value.toUpperCase();
        document.getElementById('cifrado').value = $cesar.cifrarTexto(plano, clave);
    });
    document.getElementById('btnDescifrar').addEventListener('click', function () {
        let clave = parseInt(document.getElementById('clave').value);
        let cifrado = document.getElementById('cifrado').value;
        document.getElementById('plano').value = $cesar.descifrarTexto(cifrado, clave);
    });
});
```

Un `textarea` se lee y se escribe con `.value`, igual que un `input`.

Nivel 10 · Temporizadores y el Simon (UD2E3)

Objetivo: entender que `setTimeout` y `setInterval` **no paran** el programa, y usar eso para animar una secuencia. Más una IIFE con estado (`parado` / `jugando`) separada de la interfaz.

Ejemplo resuelto 1 · Cuenta atrás (Refuerzo, ej. 15)

Qué piden: recibe dos enteros y muestra por consola del mayor al menor, uno por segundo.

Por qué no vale un `for`: el `for` termina en microsegundos; no puede "esperar". Con `setInterval` la función se ejecuta cada segundo y la paro con `clearInterval` al llegar al menor.

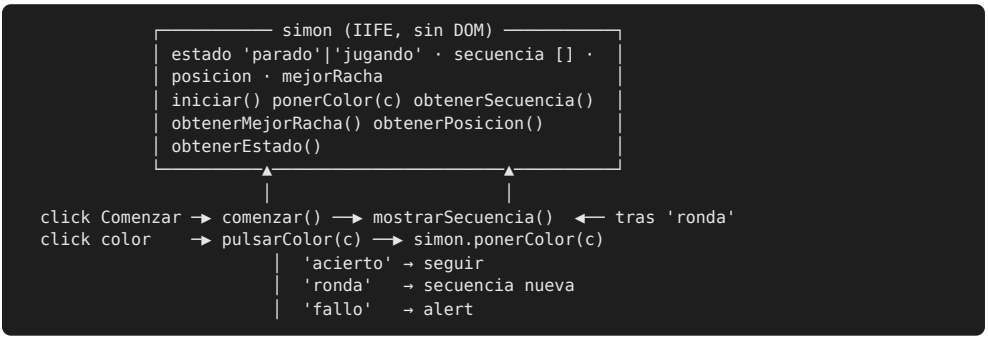
```
function cuentaAtras(a, b) {
    let mayor = a > b ? a : b;
    let menor = a > b ? b : a;
    let actual = mayor;
    let id = setInterval(function () {
        console.log(actual);
        if (actual === menor) {
            clearInterval(id);       // paro el intervalo
        }
        actual--;
    }, 1000);
}
cuentaAtras(3, 6);   // 6, 5, 4, 3 (uno por segundo)
```

`actual` vive fuera de la función del intervalo para que cada ejecución vea el valor que dejó la anterior (otro closure).

Ejemplo resuelto 2 · El Simon

Qué piden: dos columnas (juego 2×2 de colores a la izquierda; a la derecha milisegundos encendido, milisegundos entre luces, botón Comenzar, mejor racha y "5 de 7"). La lógica en una IIFE `simon` con 6 métodos: secuencia, poner color, mejor racha, posición, iniciar y estado. La interfaz separada: mostrar la secuencia, arrancar el juego y manejar la pulsación. Los botones solo tienen color mientras están encendidos. HTML, JS y CSS separados.

Trocea el problema (lo que recomienda el enunciado):



Datos de `simon`: la secuencia es un array de nombres de color (`['verde', 'azul']`) y `posicion` dice qué elemento toca comprobar ahora. Si `color === secuencia[posicion]` avanzo;

si llego al final, la ronda está completa: añado un color aleatorio y vuelvo a 0.

La parte difícil, `mostrarSecuencia`, en el tiempo (luz 600 ms, espera 300 ms, secuencia verde-verde):

```
t=0      apagar todo
t=300    encender verde        (tras la espera entre luces)
t=900    i=1 → apagar todo     (tras el tiempo encendido)
t=1200   encender verde        ← gracias a la espera se ven DOS destellos
t=1800   i=2 → apagar todo, i === length → fin, mostrandoSecuencia = false
```

Cada `setTimeout` lanza el siguiente paso: es un bucle hecho con funciones (`paso` se llama a sí misma), no con `for`. La bandera `mostrandoSecuencia` sirve para **ignorar clics** mientras se enseña la secuencia.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Simon</title>
    <link rel="stylesheet" href="simon.css">
    <script src="simon.js" defer></script>
</head>
<body>
    <div class="columnas">
        <div class="juego">
            <div class="boton verde"></div>
            <div class="boton rojo"></div>
            <div class="boton azul"></div>
            <div class="boton amarillo"></div>
        </div>
        <div>
            <div>
                <label>Milisegundos encendido:</label>
                <input type="number" id="tiempoLuz" value="600">
            </div>
            <div>
                <label>Milisegundos entre luces:</label>
                <input type="number" id="tiempoEspera" value="300">
            </div>
            <div>
                <input type="button" id="btnComenzar" value="Comenzar">
            </div>
            <div>Mejor racha: <span id="mejorRacha">0</span></div>
            <div>Aciertos: <span id="aciertos">0 de 0</span></div>
        </div>
    </div>
</body>
</html>
```

El CSS es lo mínimo para ver los colores (el Simon es el único ejercicio donde el enunciado lo exige): gris normal, color solo con la clase `encendido`.

```
.columnas { display: flex; gap: 40px; }
.juego { display: grid; grid-template-columns: 120px 120px; gap: 10px; }
.boton { height: 120px; background: gray; }
.verde.encendido { background: green; }
```

```
.rojo.encendido { background: red; }
.azul.encendido { background: blue; }
.amarillo.encendido { background: yellow; }
```

```
'use strict';

// ====================== LÓGICA DEL JUEGO ======================
const simon = (function () {
    const COLORES = ['verde', 'rojo', 'azul', 'amarillo'];
    let estado = 'parado';     // 'parado' o 'jugando'
    let secuencia = [];
    let posicion = 0;          // posición que toca comprobar
    let mejorRacha = 0;

    function colorAleatorio() {
        return COLORES[Math.floor(Math.random() * COLORES.length)];
    }

    function iniciar() {
        estado = 'jugando';
        secuencia = [colorAleatorio()];
        posicion = 0;
    }

    // Devuelve 'acierto' (sigue la ronda), 'ronda' (secuencia completa) o 'fallo'
    function ponerColor(color) {
        if (estado !== 'jugando') {
            return 'fallo';
        }
        if (color !== secuencia[posicion]) {
            estado = 'parado';
            return 'fallo';
        }
        posicion++;
        if (posicion < secuencia.length) {
            return 'acierto';
        }
        if (secuencia.length > mejorRacha) {
            mejorRacha = secuencia.length;
        }
        secuencia.push(colorAleatorio());
        posicion = 0;
        return 'ronda';
    }

    function obtenerSecuencia() {
        return secuencia;
    }

    function obtenerMejorRacha() {
        return mejorRacha;
    }

    function obtenerPosicion() {
        return posicion;
    }

    function obtenerEstado() {
        return estado;
    }
```

```
    return { obtenerSecuencia, ponerColor, obtenerMejorRacha, obtenerPosicion, iniciar,
obtenerEstado };
})();

// ====================== INTERFAZ ======================
const COLORES = ['verde', 'rojo', 'azul', 'amarillo'];
let mostrandoSecuencia = false;

function apagarTodo() {
    for (let color of COLORES) {
        document.querySelector('.boton.' + color).className = 'boton ' + color;
    }
}

function encender(color) {
    document.querySelector('.boton.' + color).className = 'boton ' + color + ' encendido';
}

function actualizarInfo() {
    document.getElementById('mejorRacha').innerHTML = simon.obtenerMejorRacha();
    document.getElementById('aciertos').innerHTML =
        `${simon.obtenerPosicion()} de ${simon.obtenerSecuencia().length}`;
}

function mostrarSecuencia() {
    let secuencia = simon.obtenerSecuencia();
    let tiempoLuz = parseInt(document.getElementById('tiempoLuz').value);
    let tiempoEspera = parseInt(document.getElementById('tiempoEspera').value);
    let i = 0;
    mostrandoSecuencia = true;

    function paso() {
        apagarTodo();                                   // 1. apagar
        if (i === secuencia.length) {                   // 5. último valor: fin
            mostrandoSecuencia = false;
            return;
        }
        setTimeout(function () {                        // 2. esperar entre luces
            encender(secuencia[i]);                     // 3. iluminar
            setTimeout(function () {                    // 4. esperar y pasar al siguiente
                i++;
                paso();
            }, tiempoLuz);
        }, tiempoEspera);
    }
    paso();
}

function comenzar() {
    if (simon.obtenerEstado() === 'jugando') {
        return;
    }
    simon.iniciar();
    actualizarInfo();
    mostrarSecuencia();
}

function pulsarColor(color) {
    if (simon.obtenerEstado() === 'parado' || mostrandoSecuencia) {
        return;
```

```
    }
    let resultado = simon.ponerColor(color);
    encender(color);
    setTimeout(apagarTodo, 200);
    actualizarInfo();
    switch (resultado) {
        case 'fallo':
            alert(`Has perdido. Mejor racha: ${simon.obtenerMejorRacha()}`);
            break;
        case 'ronda':
            mostrandoSecuencia = true;              // bloquea clics mientras espera
            setTimeout(mostrarSecuencia, 800);
            break;
    }
}

window.addEventListener('load', function () {
    document.getElementById('btnComenzar').addEventListener('click', comenzar);
    for (let color of COLORES) {
        document.querySelector('.boton.' + color).addEventListener('click', function () {
            pulsarColor(color);
        });
    }
});
```

Cómo sabe cada div su color sin id: en el `load` recorro los 4 colores y busco cada div con `querySelector('.boton.' + color)` (el que tiene las dos clases). La función anónima del evento recuerda su `color` (closure), así que el div verde siempre llama a `pulsarColor('verde')`.

Probado en el navegador: pulsar un color con el juego parado no hace nada; Comenzar enciende la secuencia; tres rondas acertadas dejan la mejor racha en 3; un fallo muestra "Has perdido. Mejor racha: 3" y el estado vuelve a `parado`.

Orden para construirlo tú:

1. HTML y CSS. En la consola: `encender('rojo')` y `apagarTodo()`.
2. La IIFE `simon`. En la consola: `simon.iniciar(); simon.obtenerSecuencia()` y `simon.ponerColor('verde')`.
3. `mostrarSecuencia` con tiempos largos (1000 ms) para verlo bien.
4. `comenzar` y `pulsarColor`. Si algo no responde, pon `console.log(resultado, simon.obtenerPosicion())` en `pulsarColor`.

Ejercicio propuesto 10 · Semáforo

Tres divs (`class="luz rojo"`, `"luz ambar"`, `"luz verde"`) que se encienden en bucle: rojo 3 s, verde 3 s, ámbar 1 s. Botón para arrancar y otro para parar.

Solución:

```
const LUCES = ['rojo', 'verde', 'ambar'];
const TIEMPOS = [3000, 3000, 1000];
let indice = 0;
let temporizador = null;

function encenderLuz(color) {
    for (let c of LUCES) {
        document.querySelector('.luz.' + c).className = 'luz ' + c;
```

```
    }
    document.querySelector('.luz.' + color).className = 'luz ' + color + ' encendido';
}

function cambiar() {
    encenderLuz(LUCES[indice]);
    temporizador = setTimeout(function () {
        indice = (indice + 1) % LUCES.length;   // 0, 1, 2, 0, 1...
        cambiar();
    }, TIEMPOS[indice]);
}

function parar() {
    clearTimeout(temporizador);
    temporizador = null;
}

window.addEventListener('load', function () {
    document.getElementById('btnArrancar').addEventListener('click', function () {
        if (temporizador === null) {
            cambiar();
        }
    });
    document.getElementById('btnParar').addEventListener('click', parar);
});
```

Nivel 11 · POO en la práctica

Objetivo: escribir clases con campos privados, getters/setters, herencia y estáticos, y saber qué pasa al usarlas. Es la base de las clases `Juguete`, `Libro`, `Autor` y `Biblioteca`.

Ejemplo resuelto 1 · El mismo Animal de dos formas

Con función constructora + prototype (forma antigua):

```
function AnimalF(nombre, sonido) {
    this.nombre = nombre;
    this.sonido = sonido;
}
AnimalF.prototype.hacerRuido = function () {
    return `${this.nombre} dice ${this.sonido}`;
};

let gato = new AnimalF('Misi', 'miau');
let perro = new AnimalF('Toby', 'guau');
gato.hacerRuido();                          // 'Misi dice miau'
gato.hacerRuido === perro.hacerRuido;       // true: comparten el método del prototype
```

Con clases y herencia:

```
class Animal {
    #nombre;
    constructor(nombre) {
        this.#nombre = nombre;
    }
    get nombre() {
```

```
        return this.#nombre;
    }
    comer() {
        return `${this.#nombre} está comiendo`;
    }
    hacerRuido() {
        return `${this.#nombre} hace un ruido`;
    }
}

class Perro extends Animal {
    #raza;
    constructor(nombre, raza) {
        super(nombre);              // el padre guarda el nombre
        this.#raza = raza;
    }
    get raza() {
        return this.#raza;
    }
    hacerRuido() {                  // sobrescribe el del padre
        return `${this.nombre} dice guau`;
    }
}

let p = new Perro('Toby', 'beagle');
p.comer();        // 'Toby está comiendo'  (heredado)
p.hacerRuido();   // 'Toby dice guau'      (sobrescrito)
p.raza;           // 'beagle'
```

Fíjate: dentro de `Perro` uso `this.nombre` (el getter del padre) y **no** `this.#nombre`. Un campo privado solo existe dentro de la clase que lo declara; ni siquiera la hija puede tocarlo.

Ejemplo resuelto 2 · Cuenta bancaria (privados, setter con validación, estático)

Qué queremos: número automático (1, 2, 3...), titular que no puede quedar vacío, saldo que solo cambia con `ingresar` y `retirar`, y un contador de cuentas creadas.

```
class Cuenta {
    static contador = 0;            // de la clase: lo comparten todas
    #numero;
    #titular;
    #saldo;

    constructor(titular, saldo = 0) {
        Cuenta.contador++;
        this.#numero = Cuenta.contador;
        this.#titular = titular;
        this.#saldo = saldo;
    }

    get numero() { return this.#numero; }     // solo getter: solo lectura
    get saldo() { return this.#saldo; }
    get titular() { return this.#titular; }
    set titular(valor) {
        if (valor && valor.trim() !== '') {
            this.#titular = valor;
        } else {
```

```
            console.log('Titular no válido');
        }
    }

    ingresar(cantidad) {
        if (cantidad <= 0) {
            return false;
        }
        this.#saldo += cantidad;
        return true;
    }

    retirar(cantidad) {
        if (cantidad <= 0 || cantidad > this.#saldo) {
            return false;
        }
        this.#saldo -= cantidad;
        return true;
    }

    static totalCuentas() {
        return Cuenta.contador;
    }
}
```

Qué pasa al usarla (probado):

Código	Resultado	Por qué
`let c1 = new Cuenta('Ana', 100)`	`c1.numero` = 1	el estático cuenta 1
`let c2 = new Cuenta('Luis')`	`c2.numero` = 2, saldo 0	parámetro por defecto
`c1.ingresar(50)`	`true`, saldo 150	
`c1.retirar(500)`	`false`, saldo 150	no hay bastante
`Cuenta.totalCuentas()`	2	se llama con la **clase**, no con `c1`
`c1.titular = ''`	"Titular no válido"	el setter valida
`c1.saldo = 1000000`	**TypeError**	solo hay getter; en clases (modo estricto) escribir en una propiedad de solo lectura da error
`c1.#saldo`	**SyntaxError**	privado

Ejemplo resuelto 3 · La clase Juguete (lo que usarás en el nivel 12)

Una clase de datos con id de solo lectura y un método que **genera su propio HTML**. Así cada objeto sabe pintarse.

```
class Juguete {
    #jugueteId;
    #nombre;
    #marca;
    #precio;

    constructor(jugueteId = 0, nombre = '', marca = '', precio = 0) {
        this.#jugueteId = jugueteId;
```

```
        this.#nombre = nombre;
        this.#marca = marca;
        this.#precio = precio;
    }

    get jugueteId() { return this.#jugueteId; }
    get nombre() { return this.#nombre; }
    set nombre(valor) { this.#nombre = valor; }
    get marca() { return this.#marca; }
    set marca(valor) { this.#marca = valor; }
    get precio() { return this.#precio; }
    set precio(valor) { this.#precio = valor; }
}
```

Los valores por defecto permiten `new Juguete()` para un juguete **vacío** (id 0 = "todavía no existe"): lo usaremos para el formulario de "Nuevo".

Ejercicio propuesto 11 · Vehículo y Coche

Clase `Vehiculo` con `#matricula` (solo lectura) y `#kilometros` (empieza en 0), método `viajar(km)` que suma si km > 0. Clase `Coche` que hereda y añade `#plazas` y un método `describir()` que devuelva `'1234ABC - 5 plazas - 120 km'`.

Solución:

```
class Vehiculo {
    #matricula;
    #kilometros = 0;
    constructor(matricula) {
        this.#matricula = matricula;
    }
    get matricula() { return this.#matricula; }
    get kilometros() { return this.#kilometros; }
    viajar(km) {
        if (km > 0) {
            this.#kilometros += km;
        }
    }
}

class Coche extends Vehiculo {
    #plazas;
    constructor(matricula, plazas) {
        super(matricula);
        this.#plazas = plazas;
    }
    describir() {
        return `${this.matricula} - ${this.#plazas} plazas - ${this.kilometros} km`;
    }
}

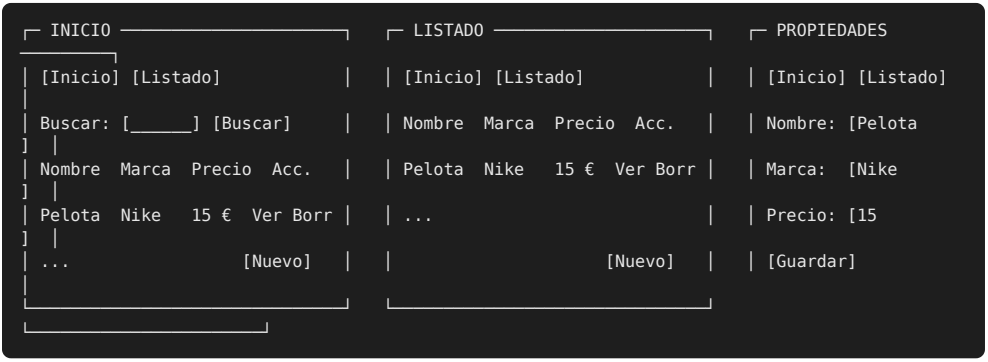
let coche = new Coche('1234ABC', 5);
coche.viajar(120);
coche.describir();   // '1234ABC - 5 plazas - 120 km'
```

Nivel 12 · La Juguetería completa, paso a paso

Objetivo: juntarlo todo en una aplicación de una sola página con **clases, campos privados, getters/setters, módulos,** `innerHTML`, `dataset`, `data-id`, `querySelector` **y** `closest`. Es el modelo de los ejercicios completos: la Biblioteca es esto mismo con tres clases.

1 · Entiende qué vas a construir (en papel)

Pantallas (todas se dibujan dentro del mismo `<div id="app">`):



Qué hace cada botón:

Botón (`data-accion`)	Dónde está	Qué hace
`inicio`	navegación	pinta la vista Inicio
`listado`	navegación	pinta la vista Listado
`buscar`	buscador	Inicio con el listado filtrado
`nuevo`	listado	Propiedades con `new Juguete()` (id 0)
`ver`	cada fila	Propiedades del juguete de esa fila
`borrar`	cada fila	pregunta, borra y vuelve al listado
`guardar`	propiedades	si id 0 crea, si no modifica; vuelve al listado

Ficheros:

```
index.html          <div id="app"> + <script type="module" src="js/main.js">
js/datos.js         array de objetos de prueba                (export default)
js/juguete.js       class Juguete: datos + generarHTMLPropiedades  (export default)
js/jugueteria.js    class Jugueteria: datos, HTML, vistas, eventos (export default)
js/main.js          en load: new Jugueteria().iniciarApp('#app')
```

2 · Orden de construcción (cada paso se prueba antes del siguiente)

1. `datos.js` y `juguete.js`. Prueba: en `main.js`, `console.log(new Juguete(1, 'a', 'b', 2).nombre)`.
2. Propiedades y constructor de `Jugueteria` (carga los datos). Prueba: `console.log(jugueteria.obtenerJuguetes())`.

3. Métodos de datos: `obtenerJuguetes(filtro)`, `obtenerJuguete(id)`, `crearJuguete(nuevo)`, `borrarJuguete(id)`. Se prueban con `console.log`, sin HTML.
4. Métodos `generarHTML…`. Solo devuelven texto: `console.log(jugueteria.generarHTMLListado(jugueteria.obtenerJuguetes()))`.
5. `iniciarApp` y las tres vistas `navegar…`. Ya se ve la página, sin eventos todavía.
6. Eventos, **de uno en uno**: inicio y listado → buscar → ver → borrar → nuevo y guardar.

3 · El código

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Juguetería</title>
    <script type="module" src="js/main.js"></script>
</head>
<body>
    <div id="app"></div>
</body>
</html>
```

```
// js/datos.js
const datos = [
    { jugueteId: 1, nombre: 'Pelota', marca: 'Nike', precio: 15 },
    { jugueteId: 2, nombre: 'Puzzle 500', marca: 'Ravensburger', precio: 12 },
    { jugueteId: 3, nombre: 'Coche teledirigido', marca: 'Hot Wheels', precio: 35 },
    { jugueteId: 4, nombre: 'Muñeca', marca: 'Barbie', precio: 20 }
];

export default datos;
```

```
// js/juguete.js
'use strict';

class Juguete {
    #jugueteId;
    #nombre;
    #marca;
    #precio;

    constructor(jugueteId = 0, nombre = '', marca = '', precio = 0) {
        this.#jugueteId = jugueteId;
        this.#nombre = nombre;
        this.#marca = marca;
        this.#precio = precio;
    }

    get jugueteId() {
        return this.#jugueteId;
    }

    get nombre() {
        return this.#nombre;
    }
```

```
    set nombre(valor) {
        this.#nombre = valor;
    }

    get marca() {
        return this.#marca;
    }

    set marca(valor) {
        this.#marca = valor;
    }

    get precio() {
        return this.#precio;
    }

    set precio(valor) {
        this.#precio = valor;
    }

    // Formulario con los datos del juguete. data-id = 0 significa juguete nuevo
    generarHTMLPropiedades() {
        return `
        <div data-componente="propiedades" data-id="${this.#jugueteId}">
            <div>
                <label>Nombre:</label>
                <input type="text" id="jg-nombre" value="${this.#nombre}">
            </div>
            <div>
                <label>Marca:</label>
                <input type="text" id="jg-marca" value="${this.#marca}">
            </div>
            <div>
                <label>Precio:</label>
                <input type="number" id="jg-precio" value="${this.#precio}">
            </div>
            <div>
                <input type="button" value="Guardar" data-accion="guardar">
            </div>
        </div>`;
    }
}

export default Juguete;
```

```
// js/jugueteria.js
'use strict';
import Juguete from './juguete.js';
import datos from './datos.js';

class Jugueteria {
    // ===== PASO 1: propiedades y constructor =====
    #contenedor;
    #juguetes;
    #contador;

    constructor() {
        this.#juguetes = [];
        this.#contador = 0;
        for (let dato of datos) {
```

```
            this.#juguetes.push(new Juguete(dato.jugueteId, dato.nombre, dato.marca,
dato.precio));
            if (dato.jugueteId > this.#contador) {
                this.#contador = dato.jugueteId;     // el siguiente id será el mayor + 1
            }
        }
    }

    // ===== PASO 2: métodos de datos (sin HTML) =====
    obtenerJuguetes(filtro) {
        if (!filtro) {
            return this.#juguetes;
        }
        let resultado = [];
        for (let juguete of this.#juguetes) {
            if (juguete.nombre.toLowerCase().includes(filtro.toLowerCase())) {
                resultado.push(juguete);
            }
        }
        return resultado;
    }

    obtenerJuguete(jugueteId) {
        for (let i in this.#juguetes) {
            if (this.#juguetes[i].jugueteId === jugueteId) {
                return this.#juguetes[i];
            }
        }
    }

    crearJuguete(nuevo) {
        this.#contador++;
        let juguete = new Juguete(this.#contador, nuevo.nombre, nuevo.marca, nuevo.precio);
        this.#juguetes.push(juguete);
        return juguete;
    }

    borrarJuguete(jugueteId) {
        for (let i in this.#juguetes) {
            if (this.#juguetes[i].jugueteId === jugueteId) {
                this.#juguetes.splice(i, 1);
                return;
            }
        }
    }

    // ===== PASO 3: métodos generarHTML (solo devuelven texto) =====
    generarHTMLNavegacion() {
        return `
        <div data-componente="navegacion">
            <input type="button" value="Inicio" data-accion="inicio">
            <input type="button" value="Listado" data-accion="listado">
        </div>`;
    }

    generarHTMLBuscador(filtro = '') {
        return `
        <div data-componente="buscador">
            <label>Buscar:</label>
            <input type="text" id="jg-filtro" value="${filtro}">
            <input type="button" value="Buscar" data-accion="buscar">
```

```
        </div>`;
    }

    generarHTMLListado(listaJuguetes) {
        let filas = '';
        for (let juguete of listaJuguetes) {
            filas += `
            <div data-id="${juguete.jugueteId}">
                <div>${juguete.nombre}</div>
                <div>${juguete.marca}</div>
                <div>${juguete.precio} €</div>
                <div>
                    <input type="button" value="Ver" data-accion="ver">
                    <input type="button" value="Borrar" data-accion="borrar">
                </div>
            </div>`;
        }
        return `
        <div data-componente="listado">
            <div>
                <div>Nombre</div><div>Marca</div><div>Precio</div><div>Acciones</div>
            </div>
            ${filas}
            <div>
                <input type="button" value="Nuevo" data-accion="nuevo">
            </div>
        </div>`;
    }

    // ===== PASO 4: arrancar y dibujar las vistas =====
    iniciarApp(selector) {
        this.#contenedor = document.querySelector(selector);
        if (!this.#contenedor) {
            alert(`No se ha encontrado el contenedor ${selector}`);
            return;
        }
        this.asignarEventos();
        this.navegarInicio();
    }

    navegarInicio(filtro = '') {
        this.#contenedor.innerHTML =
            this.generarHTMLNavegacion() +
            this.generarHTMLBuscador(filtro) +
            this.generarHTMLListado(this.obtenerJuguetes(filtro));
    }

    navegarListadoJuguetes() {
        this.#contenedor.innerHTML =
            this.generarHTMLNavegacion() +
            this.generarHTMLListado(this.obtenerJuguetes());
    }

    navegarPropiedades(juguete) {
        this.#contenedor.innerHTML =
            this.generarHTMLNavegacion() +
            juguete.generarHTMLPropiedades();
    }

    // ===== PASO 5: eventos (UN click en el contenedor para todo) =====
    asignarEventos() {
```

```
        this.#contenedor.addEventListener('click', (evento) => {
            let boton = evento.target.closest('[data-accion]');
            if (!boton) {
                return;
            }
            let fila = boton.closest('[data-id]');

            switch (boton.dataset.accion) {
                case 'inicio':
                    this.navegarInicio();
                    break;
                case 'listado':
                    this.navegarListadoJuguetes();
                    break;
                case 'buscar':
                    this.navegarInicio(this.#contenedor.querySelector('#jg-filtro').value);
                    break;
                case 'nuevo':
                    this.navegarPropiedades(new Juguete());
                    break;
                case 'ver':
                    this.navegarPropiedades(this.obtenerJuguete(Number(fila.dataset.id)));
                    break;
                case 'borrar':
                    if (prompt('¿Seguro que quieres borrarlo? (s/n)') === 's') {
                        this.borrarJuguete(Number(fila.dataset.id));
                    }
                    this.navegarListadoJuguetes();
                    break;
                case 'guardar': {
                    let nombre = this.#contenedor.querySelector('#jg-nombre').value.trim();
                    let marca = this.#contenedor.querySelector('#jg-marca').value.trim();
                    let precio = parseFloat(this.#contenedor.querySelector('#jg-
precio').value);
                    if (nombre === '' || marca === '' || isNaN(precio) || precio < 0) {
                        alert('Datos no válidos');
                        return;
                    }
                    let jugueteId = Number(fila.dataset.id);
                    if (jugueteId === 0) {
                        this.crearJuguete({ nombre: nombre, marca: marca, precio: precio });
                    } else {
                        let juguete = this.obtenerJuguete(jugueteId);
                        juguete.nombre = nombre;          // setters
                        juguete.marca = marca;
                        juguete.precio = precio;
                    }
                    this.navegarListadoJuguetes();
                    break;
                }
            }
        });
    }
}

export default Jugueteria;
```

```
// js/main.js
'use strict';
import Jugueteria from './jugueteria.js';
```

```
window.addEventListener('load', function () {
    let jugueteria = new Jugueteria();
    jugueteria.iniciarApp('#app');
});
```

4 · Entiende las cuatro ideas clave

a) Un solo `addEventListener` para toda la aplicación. Se pone en `#app`, que **nunca se borra** (lo que cambia es su `innerHTML`). Por eso se asigna una vez en `iniciarApp` y sigue funcionando en todas las vistas. Si lo pusieras en un botón, al repintar el botón sería otro y perdería el evento.

b) Recorrido de un clic en "Ver" del Puzzle:

```
clic en <input data-accion="ver">
  ↓ sube
<div data-id="2">                      ← boton.closest('[data-id]') = fila
  ↓ sube
<div data-componente="listado">
  ↓ sube
<div id="app">  ← aquí está el listener
  evento.target                = el input Ver
  closest('[data-accion]')     = el input Ver
  boton.dataset.accion         = 'ver'
  Number(fila.dataset.id)      = 2
  → navegarPropiedades(obtenerJuguete(2))
```

c) El id 0 decide si Guardar crea o modifica. "Nuevo" pinta `new Juguete()` → `data-id="0"`. "Ver" pinta un juguete existente → `data-id="2"`. En `guardar`, `boton.closest('[data-id]')` encuentra el div de propiedades y lee ese id.

d) `this` dentro del listener. El callback es una **arrow**, así que `this` sigue siendo la Juguetería y puedo llamar a `this.navegarInicio()`. Con `function (evento) { ... }` `this` sería el div y daría error.

El `case 'guardar': { ... }` lleva llaves porque declara variables con `let` dentro de un `case`; sin ellas, otro `case` con el mismo nombre de variable daría error.

Probado en el navegador: al abrir salen 4 juguetes y el buscador; buscar "PU" deja 1 (Puzzle, sin importar mayúsculas); el listado no tiene buscador; guardar un nuevo vacío avisa "Datos no válidos"; crear Peonza le da id 5; editar el precio del Puzzle a 99 se ve en el listado; borrar contestando `n` no borra y contestando `s` sí. Sin errores en consola.

5 · Si el enunciado trae `form`, `nav` y botones submit

En tu enunciado de la Juguetería los `generarHTML` venían con `<form>` y `<button type="submit">`. Funciona igual cambiando dos cosas: el buscador y las propiedades se escuchan con el evento `submit` del contenedor, y hay que llamar a `evento.preventDefault()` para que el formulario no recargue la página.

```
this.#contenedor.addEventListener('submit', (evento) => {
    evento.preventDefault();                        // sin esto la página se recarga
    switch (evento.target.dataset.componente) {     // el form que se ha enviado
        case 'buscador':
```

```
            this.navegarInicio(this.#contenedor.querySelector('#jg-filtro').value);
            break;
        case 'propiedades':
            // lo mismo que en case 'guardar'
            break;
    }
});
```

Ejercicio propuesto 12 · Añadir "Editar precio con descuento"

Añade en cada fila un botón `data-accion="rebajar"` que pida un porcentaje con `prompt`, lo valide (1-90) y baje el precio de ese juguete. Después vuelve al listado.

Solución: en `generarHTMLListado` añade `<input type="button" value="Rebajar" data-accion="rebajar">` y en el `switch`:

```
case 'rebajar': {
    let porcentaje = parseInt(prompt('Porcentaje de descuento (1-90)'));
    if (isNaN(porcentaje) || porcentaje < 1 || porcentaje > 90) {
        alert('Porcentaje no válido');
        return;
    }
    let juguete = this.obtenerJuguete(Number(fila.dataset.id));
    juguete.precio = Math.round(juguete.precio * (100 - porcentaje)) / 100;
    this.navegarListadoJuguetes();
    break;
}
```

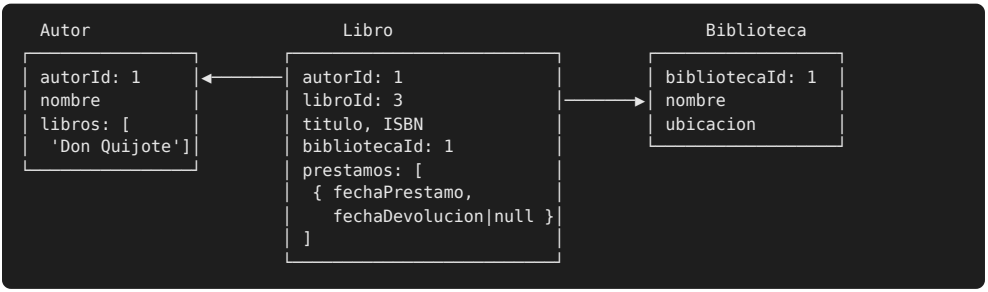
Nivel 13 · La Biblioteca: el ejercicio completo

Objetivo: construir una aplicación con **varias clases relacionadas por id**, préstamos con fechas y un gestor IIFE en módulo. Aquí tienes el núcleo resuelto y probado (libros, autores, préstamos, buscador) y, al final, lo que queda como ejercicio de examen.

1 · Qué cambia respecto a la Juguetería

Juguetería	Biblioteca	Qué tienes que hacer distinto
1 clase de datos (`Juguete`)	3 clases: `Libro`, `Autor`, `Biblioteca`	un array y un `buscar…` por clase
datos sueltos	**relaciones por id**: el libro guarda `autorId` y `bibliotecaId`	para enseñar el nombre del autor, el gestor hace `buscarAutor(libro.autorId)`
clase `Jugueteria`	IIFE `$biblio` exportada como módulo	`const $biblio = (function () { ... })();` `export default $biblio;`
—	préstamos `[{ fechaPrestamo, fechaDevolucion }]`	`estaDisponible`, `crearPrestamo`, `devolverPrestamo`, `fechaHoy()`
buscador por nombre	buscador por título **o** por autor	`buscarLibrosPorAutor` busca el autor y luego sus libros
menú dentro de `#app`	menú y buscador fijos **fuera** de `#app`	sus eventos se asignan una vez en `iniciarApp`

2 · Dibuja los datos y sus relaciones



- Un libro está **prestado** si tiene algún préstamo con `fechaDevolucion: null`.
- Si el autor guarda la lista de títulos (`libros`), al crear o borrar un libro hay que **actualizarla también** (es lo que hacían tus `anadirTituloAutor` / `quitarTituloAutor`; aquí va dentro de `crearLibro` y `borrarLibro`).

3 · Ficheros y orden de construcción

```
index.html               menú (#menu), buscador (#buscador) y <div id="app">
js/datos.js              { autores: [...], libros: [...] }
js/libro.js              class Libro      (+ generarHTMLCreacion, generarHTMLPropiedades)
js/autor.js              class Autor
js/gestorBiblioteca.js   $biblio: datos privados, buscar, crear, borrar, préstamos,
                         búsquedas, listados, iniciarApp
js/main.js               en load: $biblio.iniciarApp()
```

Orden (el mismo que seguimos al reescribir tu gestor): **clases → buscar por id → crear/borrar → préstamos → búsquedas por texto → listados → buscador → iniciarApp → eventos**. Cada paso se prueba antes del siguiente.

Truco de prueba con módulos: como el módulo no se ve desde la consola, en `main.js` pon temporalmente `window.$biblio = $biblio;`. Así en F12 puedes escribir `$biblio.buscarLibro(3).estaDisponible` o `$biblio.crearPrestamo(2)`.

4 · El código del núcleo (probado en el navegador)

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Biblioteca</title>
    <script type="module" src="js/main.js"></script>
</head>
<body>
    <div id="menu">
        <input type="button" value="Libros" data-destino="libros">
        <input type="button" value="Autores" data-destino="autores">
    </div>
    <div id="buscador">
        <label>Buscar:</label>
        <input type="text" id="buscador-texto">
        <select id="buscador-tipo">
```

```
            <option value="titulo">Por título</option>
            <option value="autor">Por autor</option>
        </select>
        <input type="button" value="Buscar" id="btnBuscar">
    </div>
    <div id="app"></div>
</body>
</html>
```

```
// js/datos.js
const datos = {
    autores: [
        { autorId: 1, nombre: 'Miguel de Cervantes', libros: ['Don Quijote', 'Novelas
ejemplares'] },
        { autorId: 2, nombre: 'Gabriel García Márquez', libros: ['Cien años de soledad'] }
    ],
    libros: [
        { libroId: 1, titulo: 'Don Quijote', ISBN: '978-1', autorId: 1, bibliotecaId: 1,
            prestamos: [{ fechaPrestamo: '2026-09-01', fechaDevolucion: '2026-09-15' }] },
        { libroId: 2, titulo: 'Novelas ejemplares', ISBN: '978-2', autorId: 1, bibliotecaId:
2, prestamos: [] },
        { libroId: 3, titulo: 'Cien años de soledad', ISBN: '978-3', autorId: 2,
bibliotecaId: 1,
            prestamos: [{ fechaPrestamo: '2026-10-10', fechaDevolucion: null }] }
    ]
};

export default datos;
```

```
// js/libro.js
'use strict';

class Libro {
    #libroId;
    #titulo;
    #ISBN;
    #autorId;
    #bibliotecaId;
    #prestamos;   // [{ fechaPrestamo, fechaDevolucion }] · fechaDevolucion null = sigue
prestado

    constructor(libroId = 0, titulo = '', ISBN = '', autorId = 0, bibliotecaId = 0, prestamos
= []) {
        this.#libroId = libroId;
        this.#titulo = titulo;
        this.#ISBN = ISBN;
        this.#autorId = autorId;
        this.#bibliotecaId = bibliotecaId;
        this.#prestamos = [];
        for (let prestamo of prestamos) {          // copia, para no tocar datos.js
            this.#prestamos.push({
                fechaPrestamo: prestamo.fechaPrestamo,
                fechaDevolucion: prestamo.fechaDevolucion ?? null   // si falta la clave →
null
            });
        }
    }
```

```
    get libroId() { return this.#libroId; }
    get titulo() { return this.#titulo; }
    set titulo(valor) { this.#titulo = valor; }
    get ISBN() { return this.#ISBN; }
    set ISBN(valor) { this.#ISBN = valor; }
    get autorId() { return this.#autorId; }
    set autorId(valor) { this.#autorId = valor; }
    get bibliotecaId() { return this.#bibliotecaId; }
    set bibliotecaId(valor) { this.#bibliotecaId = valor; }
    get prestamos() { return this.#prestamos; }

    // Disponible si NINGÚN préstamo está sin devolver
    get estaDisponible() {
        for (let prestamo of this.#prestamos) {
            if (prestamo.fechaDevolucion === null) {
                return false;      // encontrado uno sin devolver: ya lo sé
            }
        }
        return true;               // he mirado todos y ninguno está sin devolver
    }

    // Formulario de alta: el autor y la biblioteca se escriben por su id
    generarHTMLCreacion() {
        return `
        <div data-componente="libro-creacion" data-id="0">
            <div>
                <label>Título:</label>
                <input type="text" id="libro-titulo">
            </div>
            <div>
                <label>ISBN:</label>
                <input type="text" id="libro-isbn">
            </div>
            <div>
                <label>Id del autor:</label>
                <input type="number" id="libro-autor" value="${this.#autorId}">
            </div>
            <div>
                <label>Id de la biblioteca:</label>
                <input type="number" id="libro-biblioteca" value="${this.#bibliotecaId}">
            </div>
            <div>
                <input type="button" value="Crear" data-accion="crear">
            </div>
        </div>`;
    }

    generarHTMLPropiedades() {
        let disponible = 'No';
        let botonPrestamo = '<input type="button" value="Devolver" data-accion="devolver">';
        if (this.estaDisponible) {
            disponible = 'Sí';
            botonPrestamo = '<input type="button" value="Prestar" data-accion="prestar">';
        }
        let filasPrestamos = '';
        for (let prestamo of this.#prestamos) {
            filasPrestamos += `
            <div>
                <div>${prestamo.fechaPrestamo}</div>
                <div>${prestamo.fechaDevolucion ?? 'Sin devolver'}</div>
            </div>`;
```

```
        }
        return `
        <div data-componente="libro-propiedades" data-id="${this.#libroId}">
            <div>Título: ${this.#titulo}</div>
            <div>ISBN: ${this.#ISBN}</div>
            <div>Id del autor: ${this.#autorId}</div>
            <div>Id de la biblioteca: ${this.#bibliotecaId}</div>
            <div>Disponible: ${disponible}</div>
            <div>
                ${botonPrestamo}
                <input type="button" value="Borrar" data-accion="borrar">
            </div>
            <div>
                <div><div>Fecha préstamo</div><div>Fecha devolución</div></div>
                ${filasPrestamos}
            </div>
        </div>`;
    }
}

export default Libro;
```

```
// js/autor.js
'use strict';

class Autor {
    #autorId;
    #nombre;
    #libros;   // títulos de sus libros

    constructor(autorId = 0, nombre = '', libros = []) {
        this.#autorId = autorId;
        this.#nombre = nombre;
        this.#libros = [];
        for (let titulo of libros) {
            this.#libros.push(titulo);
        }
    }

    get autorId() { return this.#autorId; }
    get nombre() { return this.#nombre; }
    set nombre(valor) { this.#nombre = valor; }
    get libros() { return this.#libros; }
}

export default Autor;
```

```
// js/gestorBiblioteca.js
'use strict';
import Libro from './libro.js';
import Autor from './autor.js';
import datos from './datos.js';

const $biblio = (function () {
    // ===== PASO 1: datos privados (cargados desde datos.js) =====
    let contenedor;
    let libros = [];
    let autores = [];
```

```
    let contadorLibros = 0;

    for (let dato of datos.autores) {
        autores.push(new Autor(dato.autorId, dato.nombre, dato.libros));
    }
    for (let dato of datos.libros) {
        libros.push(new Libro(dato.libroId, dato.titulo, dato.ISBN, dato.autorId,
dato.bibliotecaId, dato.prestamos));
        if (dato.libroId > contadorLibros) {
            contadorLibros = dato.libroId;
        }
    }

    function fechaHoy() {
        let hoy = new Date();
        let mes = hoy.getMonth() + 1;
        let dia = hoy.getDate();
        if (mes < 10) mes = '0' + mes;
        if (dia < 10) dia = '0' + dia;
        return hoy.getFullYear() + '-' + mes + '-' + dia;
    }

    // ===== PASO 2: buscar por id =====
    function buscarLibro(libroId) {
        for (let i in libros) {
            if (libros[i].libroId === libroId) {
                return libros[i];
            }
        }
    }

    function buscarAutor(autorId) {
        for (let i in autores) {
            if (autores[i].autorId === autorId) {
                return autores[i];
            }
        }
    }

    // ===== PASO 3: crear y borrar =====
    function crearLibro(nuevo) {
        let autor = buscarAutor(nuevo.autorId);
        if (!autor) {
            alert('No existe un autor con ese id');
            return;
        }
        contadorLibros++;
        let libro = new Libro(contadorLibros, nuevo.titulo, nuevo.ISBN, nuevo.autorId,
nuevo.bibliotecaId);
        libros.push(libro);
        autor.libros.push(libro.titulo);          // el autor también apunta el título
        return libro;
    }

    function borrarLibro(libroId) {
        for (let i in libros) {
            if (libros[i].libroId === libroId) {
                let autor = buscarAutor(libros[i].autorId);
                if (autor) {
                    let posicion = autor.libros.indexOf(libros[i].titulo);
                    if (posicion !== -1) {
```

```
                        autor.libros.splice(posicion, 1);   // splice, no slice
                    }
                }
                libros.splice(i, 1);
                return;
            }
        }
    }

    // ===== PASO 4: préstamos =====
    function crearPrestamo(libroId) {
        let libro = buscarLibro(libroId);
        if (!libro.estaDisponible) {
            alert('El libro ya está prestado');
            return;
        }
        libro.prestamos.push({ fechaPrestamo: fechaHoy(), fechaDevolucion: null });
    }

    function devolverPrestamo(libroId) {
        let libro = buscarLibro(libroId);
        for (let prestamo of libro.prestamos) {
            if (prestamo.fechaDevolucion === null) {
                prestamo.fechaDevolucion = fechaHoy();   // prestamo. delante
                return;
            }
        }
        alert('El libro no está prestado');
    }

    // ===== PASO 5: búsquedas por texto =====
    function buscarLibrosPorTitulo(texto) {
        let resultado = [];
        for (let libro of libros) {
            if (libro.titulo.toLowerCase().includes(texto.toLowerCase())) {
                resultado.push(libro);
            }
        }
        return resultado;
    }

    function buscarLibrosPorAutor(texto) {
        let resultado = [];
        for (let autor of autores) {
            if (autor.nombre.toLowerCase().includes(texto.toLowerCase())) {
                for (let libro of libros) {
                    if (libro.autorId === autor.autorId) {
                        resultado.push(libro);
                    }
                }
            }
        }
        return resultado;
    }

    // ===== PASO 6: listados (el gestor resuelve el nombre del autor) =====
    function generarHTMLListadoLibros(lista = libros) {
        let filas = '';
        for (let libro of lista) {
            let autor = buscarAutor(libro.autorId);
            filas += `
```

```
            <div data-id="${libro.libroId}">
                <div>${libro.titulo}</div>
                <div>${autor ? autor.nombre : 'Desconocido'}</div>
                <div>${libro.estaDisponible ? 'Disponible' : 'Prestado'}</div>
                <div>
                    <input type="button" value="Ver" data-accion="ver">
                    <input type="button" value="Borrar" data-accion="borrar">
                </div>
            </div>`;
        }
        return `
        <div data-componente="listado-libros">
            <div><div>Título</div><div>Autor</div><div>Estado</div><div>Acciones</div></div>
            ${filas}
            <div data-id="0">
                <input type="button" value="Nuevo libro" data-accion="nuevo">
            </div>
        </div>`;
    }

    function generarHTMLListadoAutores() {
        let filas = '';
        for (let autor of autores) {
            filas += `
            <div data-id="${autor.autorId}">
                <div>${autor.nombre}</div>
                <div>${autor.libros.length} libros</div>
            </div>`;
        }
        return `<div data-componente="listado-autores">${filas}</div>`;
    }

    function generarHTMLResultadoBuscador(tipo, texto) {
        if (tipo === 'autor') {
            return generarHTMLListadoLibros(buscarLibrosPorAutor(texto));
        }
        return generarHTMLListadoLibros(buscarLibrosPorTitulo(texto));
    }

    // ===== PASO 7: pintar y eventos =====
    function mostrar(html) {
        contenedor.innerHTML = html;
    }

    function iniciarApp() {
        contenedor = document.querySelector('#app');
        if (!contenedor) {
            alert('No se ha encontrado el contenedor #app');
            return;
        }

        // Menú y buscador están FUERA de #app: sus eventos se asignan una vez
        document.querySelector('#menu').addEventListener('click', (evento) => {
            let boton = evento.target.closest('[data-destino]');
            if (!boton) {
                return;
            }
            switch (boton.dataset.destino) {
                case 'libros':
                    mostrar(generarHTMLListadoLibros());
                    break;
```

```
                case 'autores':
                    mostrar(generarHTMLListadoAutores());
                    break;
            }
        });

        document.querySelector('#btnBuscar').addEventListener('click', () => {
            let tipo = document.querySelector('#buscador-tipo').value;
            let texto = document.querySelector('#buscador-texto').value;
            mostrar(generarHTMLResultadoBuscador(tipo, texto));
        });

        // Todo lo que se pinta dentro de #app: un solo click
        contenedor.addEventListener('click', (evento) => {
            let boton = evento.target.closest('[data-accion]');
            if (!boton) {
                return;
            }
            // la fila del listado o la ficha del libro llevan el data-id
            let libroId = Number(boton.closest('[data-id]').dataset.id);

            switch (boton.dataset.accion) {
                case 'nuevo':
                    mostrar(new Libro().generarHTMLCreacion());
                    break;
                case 'crear': {
                    let titulo = contenedor.querySelector('#libro-titulo').value.trim();
                    let ISBN = contenedor.querySelector('#libro-isbn').value.trim();
                    let autorId = parseInt(contenedor.querySelector('#libro-autor').value);
                    let bibliotecaId = parseInt(contenedor.querySelector('#libro-
biblioteca').value);
                    if (titulo === '' || isNaN(autorId) || isNaN(bibliotecaId)) {
                        alert('Datos no válidos');
                        return;
                    }
                    if (crearLibro({ titulo: titulo, ISBN: ISBN, autorId: autorId,
bibliotecaId: bibliotecaId })) {
                        mostrar(generarHTMLListadoLibros());
                    }
                    break;
                }
                case 'ver':
                    mostrar(buscarLibro(libroId).generarHTMLPropiedades());
                    break;
                case 'borrar':
                    if (prompt('¿Seguro que quieres borrarlo? (s/n)') === 's') {
                        borrarLibro(libroId);
                        mostrar(generarHTMLListadoLibros());
                    }
                    break;
                case 'prestar':
                    crearPrestamo(libroId);
                    mostrar(buscarLibro(libroId).generarHTMLPropiedades());
                    break;
                case 'devolver':
                    devolverPrestamo(libroId);
                    mostrar(buscarLibro(libroId).generarHTMLPropiedades());
                    break;
            }
        });
```

```
        mostrar(generarHTMLListadoLibros());
    }

    // ===== Interfaz pública =====
    return {
        iniciarApp,
        buscarLibro,
        buscarAutor,
        crearLibro,
        borrarLibro,
        crearPrestamo,
        devolverPrestamo,
        buscarLibrosPorTitulo,
        buscarLibrosPorAutor,
        generarHTMLListadoLibros,
        generarHTMLListadoAutores,
        generarHTMLResultadoBuscador
    };
})();

export default $biblio;
```

```
// js/main.js
'use strict';
import $biblio from './gestorBiblioteca.js';

window.addEventListener('load', function () {
    $biblio.iniciarApp();
    window.$biblio = $biblio;   // SOLO para probar desde la consola F12; quítalo al acabar
});
```

Probado en el navegador: el listado muestra el nombre del autor (resuelto por id) y "Prestado" en Cien años de soledad; en su ficha, Devolver pone la fecha de hoy y el botón cambia a Prestar; Prestar añade un préstamo "Sin devolver"; buscar "cervantes" por autor da sus 2 libros; buscar "SOLEDAD" por título da 1; crear un libro con autor 99 avisa "No existe un autor con ese id"; con autor 1 se crea y el autor pasa a tener sus títulos actualizados; borrar Don Quijote quita también su título del autor.

5 · Entiende los puntos nuevos

`estaDisponible` **devuelve** `false` **dentro del bucle y** `true` **después.** No puedo decir "disponible" hasta haber mirado **todos** los préstamos; pero en cuanto encuentro uno sin devolver, ya sé que no lo está. Es exactamente `esPrimo` del nivel 4.

`fechaDevolucion: prestamo.fechaDevolucion ?? null`**.** Si en `datos.js` un préstamo no trae la clave `fechaDevolucion`, valdría `undefined`, y `undefined === null` es `false`: el libro saldría disponible sin estarlo. Con `??` lo convierto a `null`. Fue uno de tus fallos.

El gestor resuelve el nombre del autor, no la clase `Libro`**.** Los `generarHTML` de las clases **no reciben parámetros** y un libro no conoce la lista de autores; por eso la ficha del libro enseña el id, y el listado (que hace el gestor) enseña el nombre con `buscarAutor(libro.autorId)`.

`data-id="0"` **en el botón "Nuevo".** El listener lee siempre `boton.closest('[data-id]')`. Para que no falle con el botón Nuevo (que no está en ninguna fila), lo meto en un div con `data-`

`id="0"`.

`if (crearLibro(...))`. `crearLibro` devuelve el libro si lo crea y `undefined` si el autor no existe. Así, solo cambio de pantalla si ha ido bien.

6 · Ejercicio final (tipo examen): completa la Biblioteca

Con lo anterior como base, añade sin mirar las soluciones:

- ☐ Clase `Biblioteca` (`bibliotecaId`, `nombre`, `ubicacion`) con `generarHTMLCreacion()` y `generarHTMLEdicion()`, su array en `datos.js` y `buscarBiblioteca(id)` en el gestor.
- ☐ Menú "Bibliotecas" con `generarHTMLListadoBibliotecas()`.
- ☐ Crear y borrar autor. Un autor con libros no se puede borrar.
- ☐ Editar libro: `generarHTMLEdicion()` con los valores actuales y un botón `data-accion="guardar"` que use los setters.
- ☐ En el listado de libros, mostrar también el nombre de la biblioteca.

Soluciones de las piezas clave:

```
// En el gestor: crear y borrar autor
let contadorAutores = 2;   // el mayor id de datos.autores

function crearAutor(nuevo) {
    contadorAutores++;
    let autor = new Autor(contadorAutores, nuevo.nombre);
    autores.push(autor);
    return autor;
}

function borrarAutor(autorId) {
    for (let libro of libros) {
        if (libro.autorId === autorId) {
            alert('No se puede borrar: el autor tiene libros');
            return;
        }
    }
    for (let i in autores) {
        if (autores[i].autorId === autorId) {
            autores.splice(i, 1);
            return;
        }
    }
}
```

```
// En Libro: formulario de edición (sin parámetros, con sus valores)
generarHTMLEdicion() {
    return `
    <div data-componente="libro-edicion" data-id="${this.#libroId}">
        <div>
            <label>Título:</label>
            <input type="text" id="libro-titulo" value="${this.#titulo}">
        </div>
        <div>
            <label>ISBN:</label>
            <input type="text" id="libro-isbn" value="${this.#ISBN}">
        </div>
```

```
        <div>
            <input type="button" value="Guardar" data-accion="guardar">
        </div>
    </div>`;
}

// En el switch del gestor
case 'guardar': {
    let libro = buscarLibro(libroId);
    let titulo = contenedor.querySelector('#libro-titulo').value.trim();
    if (titulo === '') {
        alert('El título es obligatorio');
        return;
    }
    let autor = buscarAutor(libro.autorId);
    let posicion = autor.libros.indexOf(libro.titulo);   // el autor guarda el título:
    if (posicion !== -1) {
        autor.libros[posicion] = titulo;                  // lo cambio también ahí
    }
    libro.titulo = titulo;
    libro.ISBN = contenedor.querySelector('#libro-isbn').value.trim();
    mostrar(libro.generarHTMLPropiedades());
    break;
}
```

Para la columna de biblioteca en el listado, igual que con el autor: `let biblioteca = buscarBiblioteca(libro.bibliotecaId);` y `${biblioteca ? biblioteca.nombre : 'Desconocida'}`.

PARTE III · ANTES DEL EXAMEN

Errores típicos (los que salieron en tus ejercicios)

Cuando algo no funciona, mira primero esta tabla: casi todos los fallos que hemos depurado juntos son de uno de estos tipos.

Síntoma	Causa	Arreglo
La figura sale `NaN`	`linea *= '*'`	`linea += '*'`
La figura sale torcida en todas las filas	`n - 1` en vez de `n - fila`	la fórmula depende de la fila
La figura se ve rara en el alert	el alert usa letra proporcional	compruébala con `console.log`
`Cannot read properties of undefined (reading 'length')`	`lista.lenght` o `categoría` vs `categoria`	revisa la ortografía exacta
El `for...in` no encuentra nada	usas otra variable en vez del índice (`nombre` en vez de `i`)	`for (let i in lista) { lista[i] ... }`
Salen 5 alerts "no existe"	el `alert` está **dentro** del bucle de búsqueda	busca primero, decide fuera del bucle
En la página aparece el código de una función	falta `()` al llamarla (`this.generarHTMLBuscador`)	`this.generarHTMLBuscador()`
Un botón no hace nada	`data-accion="guardad"` no coincide con `case 'guardar'`	mismo texto en el HTML y en el `case` ; `console.log(boton.dataset.accion)`
`obtenerJuguete(fila.dataset.id)` da `undefined`	`dataset` da texto y comparas con `===` un número	`Number(fila.dataset.id)`
Borrar no borra (o borra mal)	`slice` en vez de `splice`	`lista.splice(posicion, 1)`
`Cannot read properties of undefined (reading 'push')`	`libros.prestamos.push` (el array) en vez de `libro.prestamos.push`	singular: el objeto concreto
Devolver no cambia nada	`fechaDevolucion = fechaHoy()` sin `prestamo.` delante	`prestamo.fechaDevolucion = fechaHoy()`
Un getter devuelve `undefined`	declaraste un campo público con el mismo nombre (`titulo;`)	arriba solo los privados `#titulo;`
El libro sale disponible estando prestado	el préstamo no trae `fechaDevolucion` (vale `undefined`)	`prestamo.fechaDevolucion ?? null`
Una búsqueda por id da `undefined`	los datos empiezan en otro número (101…)	mira `datos.js` antes de probar
Cannot use import statement outside a module	falta `type="module"`	`<script type="module" src="js/main.js">`
does not provide an export named default	falta `export default` o importas con llaves algo que es default	`export default Juguete;` → `import Juguete from ...`
404 al cargar un .js	la ruta o la carpeta no coinciden	`./js/juguete.js` , mira la pestaña Red
`null.addEventListener`	el script corre antes de que exista el HTML	asigna eventos en `load` o usa `defer`

Síntoma	Causa	Arreglo
Los eventos dejan de funcionar al repintar	el evento estaba en un elemento que `innerHTML` ha sustituido	pon el evento en el contenedor que no se borra
`this.navegarInicio is not a function` dentro de un evento	callback con `function` en vez de arrow	`(evento) => { ... }`
`'1' + 1` da `'11'`	el valor vino de `prompt`, `value`, `dataset` o `for...in`	`parseInt`, `parseFloat` o `Number` antes de operar

Cómo depurar en 4 pasos: (1) lee el error rojo de la consola y su línea; (2) pon `console.log` antes de esa línea con las variables que usa; (3) comprueba el HTML generado en la pestaña Elementos (ids, `data-id`, `data-accion`); (4) arregla una cosa cada vez y vuelve a probar.

Checklist para el examen

Lo que tienes que saber escribir sin mirar

- ☐ Pedir un número válido con `do...while` + `parseInt` + `isNaN`.
- ☐ Menú con `do...while` + `switch` y opción 0 para salir.
- ☐ Aleatorio entre min y max: `Math.floor(Math.random() * (max - min + 1)) + min`.
- ☐ Una figura: tabla fila/espacios/asteriscos y dos bucles dentro del de filas.
- ☐ Buscar en un array de objetos con `for...in` y devolver el objeto, o la posición / -1.
- ☐ Borrar con `splice(posicion, 1)`; ordenar con `toSorted((a, b) => a.campo - b.campo)` o `localeCompare`.
- ☐ Contar o agrupar con un objeto: ¿existe? → créalo → úsalo.
- ☐ Closure que devuelve una función; IIFE con datos privados y `return { ... }`.
- ☐ `JSON.stringify`, `JSON.parse` dentro de `try...catch` y `Array.isArray`.
- ☐ Clase con `#privados`, constructor con valores por defecto, getters/setters, `static`, `extends` + `super`.
- ☐ `export default` / `import` y `<script type="module">`.
- ☐ Página: `getElementById(...).value`, `innerHTML`, `addEventListener` en `load`.
- ☐ Delegación: un `click` en el contenedor, `closest('[data-accion]')`, `closest('[data-id]')`, `Number(dataset.id)`, `switch`.
- ☐ `setTimeout` encadenado y `setInterval` + `clearInterval`.

Plantilla de arranque para un ejercicio completo

Cuando te den un enunciado tipo Juguetería o Biblioteca, empieza siempre así:

1. **Lee** el enunciado entero y subraya: entidades (clases), datos de cada una, métodos que piden **con su nombre exacto**, pantallas y botones.
2. **Dibuja** las pantallas y la relación entre clases (qué id guarda cada una).
3. **Escribe** `datos.js` con 3 o 4 objetos de prueba.
4. **Clases** con privados, constructor con valores por defecto, getters/setters y sus `generarHTML` sin parámetros. Prueba en consola.

5. **Gestor**: datos → buscar por id → crear → borrar → resto de lógica. Prueba en consola cada función.
6. `generarHTML` **del gestor** y `mostrar(html)` / `navegar…`.
7. **Eventos**, de uno en uno, con `console.log(boton.dataset.accion)` mientras pruebas.
8. Solo los métodos que pide el enunciado; comenta cada bloque.

Autoevaluación rápida: ¿qué muestra?

Código	Respuesta
`console.log('5' + 3)`	`'53'`
`console.log('5' - 3)`	`2`
`console.log(typeof null)`	`'object'`
`console.log(0 || 'x', 0 ?? 'x')`	`x 0`
`let a = [3, 20, 100]; a.sort(); console.log(a)`	`[100, 20, 3]` (orden de texto)
`for (let i in ['a', 'b']) console.log(i + 1)`	`'01'` y `'11'` (i es texto)
`let f = crearContador(); f(); console.log(f())`	`2`
`console.log([1, 2, 3].slice(1))` y el array después	`[2, 3]` ; el array sigue `[1, 2, 3]`
`let c = 5; console.log(c++, c)`	`5 6`
`console.log(Math.floor(-4.5), Math.trunc(-4.5))`	`-5 -4`
`const o = { n: 1 }; o.n = 2;` ¿error?	No: `const` impide reasignar `o`, no cambiar sus propiedades
`class A { #x = 1 } new A().#x`	Error: campo privado
`setTimeout(() => console.log('B'), 0); console.log('A');`	`A` y luego `B`

El día del examen

- Pon `'use strict';` arriba y abre F12 desde el primer minuto.
- Haz que **algo** funcione pronto (cargar y listar datos) y ve añadiendo; una aplicación que lista y crea vale más que una que lo intenta todo y no arranca.
- Después de cada función nueva, guarda (Ctrl+S) y prueba. Un error a la vez.
- Respeta los nombres del enunciado (`crearPrestamo` , no `prestar`).
- Antes de entregar: quita los `console.log` de prueba y el `window.$biblio` si lo pusiste.